

LYTA — Onboarding Développeur

LYTA — Onboarding Développeur

Version 1.0 — 8 juin 2026 **Audience** Développeur externe missionné pour auditer et améliorer la plateforme LYTA **Statut** Vivant — sera mis à jour à chaque évolution majeure **Document associé (confidentiel)** `LYTA_Security_Audit_CONFIDENTIAL.md` — remis séparément avec NDA renforcé

Avertissement & cadre d'utilisation

Ce document est **confidentiel** et destiné exclusivement au développeur signataire du NDA conclu avec **Optimislisk Sàrl** (entité propriétaire de LYTA).

Périmètre autorisé

-  Lecture du code source LYTA fourni en accès lecture/écriture sur le repo Git privé
-  Accès **administrateur** au tenant `advisy.lyta.ch` (production, données réelles de clients du cabinet Advisy)
-  Accès **administrateur** au compte Supabase du projet (lecture des migrations, RLS, edge functions)
-  Possibilité de proposer des refactorings, audits, optimisations
-  **Aucune exfiltration de données clients** (noms, AVS, IBAN, dossiers santé) hors du périmètre technique
-  **Aucune modification en production** sans validation explicite préalable de Habib Agharbi
-  **Aucun partage** de ce document, des credentials ou du code avec un tiers

Cadre juridique applicable

Cadre	Implication
nLPD (Suisse)	Les clients Advisy sont des personnes physiques résidentes suisses → traitement de données personnelles soumis à la nLPD du 1 ^{er} septembre 2023. Le dev agit comme sous-traitant au sens de la loi.
FINMA / Secret professionnel	Advisy conseille en assurance — données soumises au secret professionnel de l'art. 47 LB / OS-FINMA. Le dev est lié par cette confidentialité dès lors qu'il a accès à des dossiers clients.
RGPD	Si le dev est résident UE ou si LYTA traite des données de résidents UE, le RGPD s'applique en complément (DPA, base légale, finalité, durée).
Responsabilité civile	Toute fuite ou modification non autorisée engage la responsabilité du dev vis-à-vis d'Optimislink.

Note opérationnelle Habib : *Tu m'as confirmé assumer le risque d'un accès admin direct sur advisy plutôt que de passer par un tenant DEMO. Cette décision est tracée dans la conversation du 8 juin 2026 et dans ce document. Je te recommande très fortement de faire signer au dev (a) un NDA explicite, (b) un Data Processing Agreement (DPA) listant les finalités du traitement, (c) une clause de destruction des données à la fin de la mission.*

Table des matières

1. [Vue d'ensemble produit](#)
2. [Stack & versions](#)
3. [Architecture globale](#)
4. [Modèle de données](#)
5. [Multi-tenancy & permissions](#)
6. [Modules métier](#)
7. [Edge Functions catalogue](#)
8. [Auth & Permissions \(détail technique\)](#)
9. [Déploiement](#)
10. [Conventions code](#)
11. [Dette technique & bugs UX connus](#)
12. [Roadmap](#)
13. [Annexes — incluant §13.4 Docs existantes et §13.5 Détails opérationnels concrets](#)

1. Vue d'ensemble produit

1.1 Qu'est-ce que LYTA ?

LYTA est une plateforme SaaS multi-tenant à destination des **cabinets de courtage en assurance suisses** (et, à terme, à d'autres marchés). Elle outille les courtiers indépendants et les cabinets sur l'intégralité de leur métier :

- **Gestion clients** : carnet d'adresses, membres de famille, documents, suivis
- **Gestion contrats / polices** : portefeuille, échéances, primes, sinistres
- **Smartflow** : scan + classification IA de documents (polices, décomptes, attestations)
- **Mandats de gestion** : génération + signature à distance + dispatch automatisé aux compagnies
- **Commissions** : règles tarifaires par compagnie/produit, calcul, rétrocessions, décomptes
- **Communications** : emailing transactionnel, campagnes, SMS, notifications
- **Espace client** : portail propre à chaque client final (lecture seule + signature de docs)
- **Compareur** (en préparation, voir Optimis) : moteur de comparaison de produits d'assurance

1.2 Modèle économique

- **SaaS B2B abonnement mensuel/annuel** géré via **Stripe**
- Tarification par **seats** (collaborateurs CRM) — voir `useTenantSeats.tsx` et `apply-monthly-coverage`
- **Plans** : configurés dans la table `platform_plans` (Starter / Pro / Enterprise...)
- **Quotas** : par tenant et par module (ex: nombre de clients, nombre de contrats, nombre de scans IA/mois)
- **Modules activables** : `plan_modules` × `platform_modules` (un plan donne accès à un sous-ensemble de modules)
- **Règle critique facturation** : les `extra_users` correspondent aux **collaborateurs CRM uniquement**. Les **clients du portail espace-client sont illimités et gratuits**. Voir `lyta_pricing_users.md` dans la mémoire projet.

1.3 Acteurs & rôles

LYTA distingue trois grands types d'utilisateurs :

Rôle	Description	Espace
KING (Optimislink)	Administrateur de la plateforme (Habib) — gestion des tenants, plans, facturation, support, monitoring	<code>app.lyta.ch/king/*</code>
Collaborateur tenant	Membre d'un cabinet (admin, manager, agent)	<code><tenant>.lyta.ch/crm/*</code> (sous-domaine)
Client final	Client du cabinet, accède à son espace personnel	<code><tenant>.lyta.ch/client/*</code>

Détail rôles **collaborateurs** : - **Admin** : accès complet au CRM du cabinet - **Manager** : voit ses agents + ses propres clients - **Agent** : voit uniquement ses clients assignés (scope `personal` ou `team`)

Détail des scopes dans la section [5. Multi-tenancy & permissions](#).

1.4 URLs en production

Environnement	URL
App principale	<code>https://app.lyta.ch</code>
Sous-domaines tenants	<code>https://<slug>.lyta.ch</code> (ex: <code>advisy.lyta.ch</code> , <code>jcgconsulting.lyta.ch</code>)
Backup Vercel	<code>https://lyta-xi.vercel.app</code>
API Supabase	<code>https://shxbcszukoegvvejcpn.supabase.co</code>
Edge Functions	<code>https://shxbcszukoegvvejcpn.supabase.co/functions/v1/<name></code>
Storage	bucket <code>documents</code> (privé, RLS strict)

1.5 État actuel (juin 2026)

- **Tenants actifs en production** : ~3-5 (dont `advisy` , `jcgconsulting`)
- **Stack vivante** depuis ~12 mois
- **Roadmap** : voir `Documentation/Roadmap_LYTA_Officielle.md` (vivant, 6 priorités)
- **Dette technique** : présente mais cartographiée — voir [section 11](#)

2. Stack & versions

2.1 Frontend

Domaine	Lib / framework	Version	Rôle
Build / dev	Vite	5.4.19	Bundler, dev server, HMR
Compilation	@vitejs/plugin-react-swc	3.11	SWC pour TS/JSX (plus rapide que Babel)
Lang	TypeScript	5.8.3	Strict mode, types Supabase auto-générés
UI core	React	18.3.1	Functional components + hooks
Routing	react-router-dom	6.30.1	Routing déclaratif avec layouts imbriqués
State serveur	@tanstack/react-query	5.83	Cache des queries Supabase, invalidation, optimistic updates
Forms	react-hook-form + zod	7.61 / 3.25	Validation déclarative
Styling	Tailwind CSS + tailwind-merge + CVA	3.4 / 2.6 / 0.7	Classes utilitaires + variantes
Composants	shadcn/ui sur Radix UI	radix 1.x-2.x	Primitives accessibles non-stylisées
Animations	framer-motion	12.23	Animations + transitions
Icônes	lucide-react	0.462	Pack d'icônes SVG
Toasts	sonner + Radix toast	1.7 / 1.2	Notifications
Tableaux	recharts	2.15	Graphiques (dashboards, rapports)
3D	three + @react-three/fiber + drei	0.160 / 8.18 / 9.122	Effets visuels marketing (peu utilisé en CRM)
PDF	pdf-lib + pdfjs-dist	1.17 / 4.7.76	Génération PDF (mandat signé) + affichage PDF (signature à distance)
QR codes	qrcode	1.5	Génération QR-factures suisses
XLSX	xlsx	0.18	Import/export de listes (clients, commissions)
Date	react-day-picker	8.10	DatePicker UI
Sanitization	dompurify	3.3	Cleanup HTML user-generated (anti-XSS)
Theme	next-themes	0.3	Mode clair/sombre
Carousel	embla-carousel-react + autoplay	8.6	Onboarding tour, marketing

Domaine	Lib / framework	Version	Rôle
OTP input	input-otp	1.4	Saisie SMS 2FA

2.2 Backend

Domaine	Tech	Version
Database	PostgreSQL (Supabase managé)	15.x
Auth	Supabase Auth (GoTrue)	géré Supabase
API	PostgREST auto-généré + RPC functions PL/pgSQL	géré Supabase
Edge runtime	Deno (Supabase Edge Functions)	Deno 1.x
Storage	Supabase Storage (S3-compatible)	géré Supabase
Realtime	Supabase Realtime (postgres_changes / broadcast)	géré Supabase
SDK client	@supabase/supabase-js	2.81

2.3 Services tiers

Service	Usage
Resend	Email transactionnel (invitations, notifications, dispatch mandat aux compagnies)
Twilio Verify	Vérification SMS (2FA, validation email pour invitations)
Stripe	Facturation abonnement tenants, webhooks
OpenAI / Anthropic	Classification IA des documents scannés (Smartflow)
OpenPLZ / Swiss-post	Validation codes postaux suisses, lookup d'adresses
Vercel	Hosting frontend (auto-deploy sur push GitHub)
GitHub	Repo privé LYTA-git

2.4 Mobile

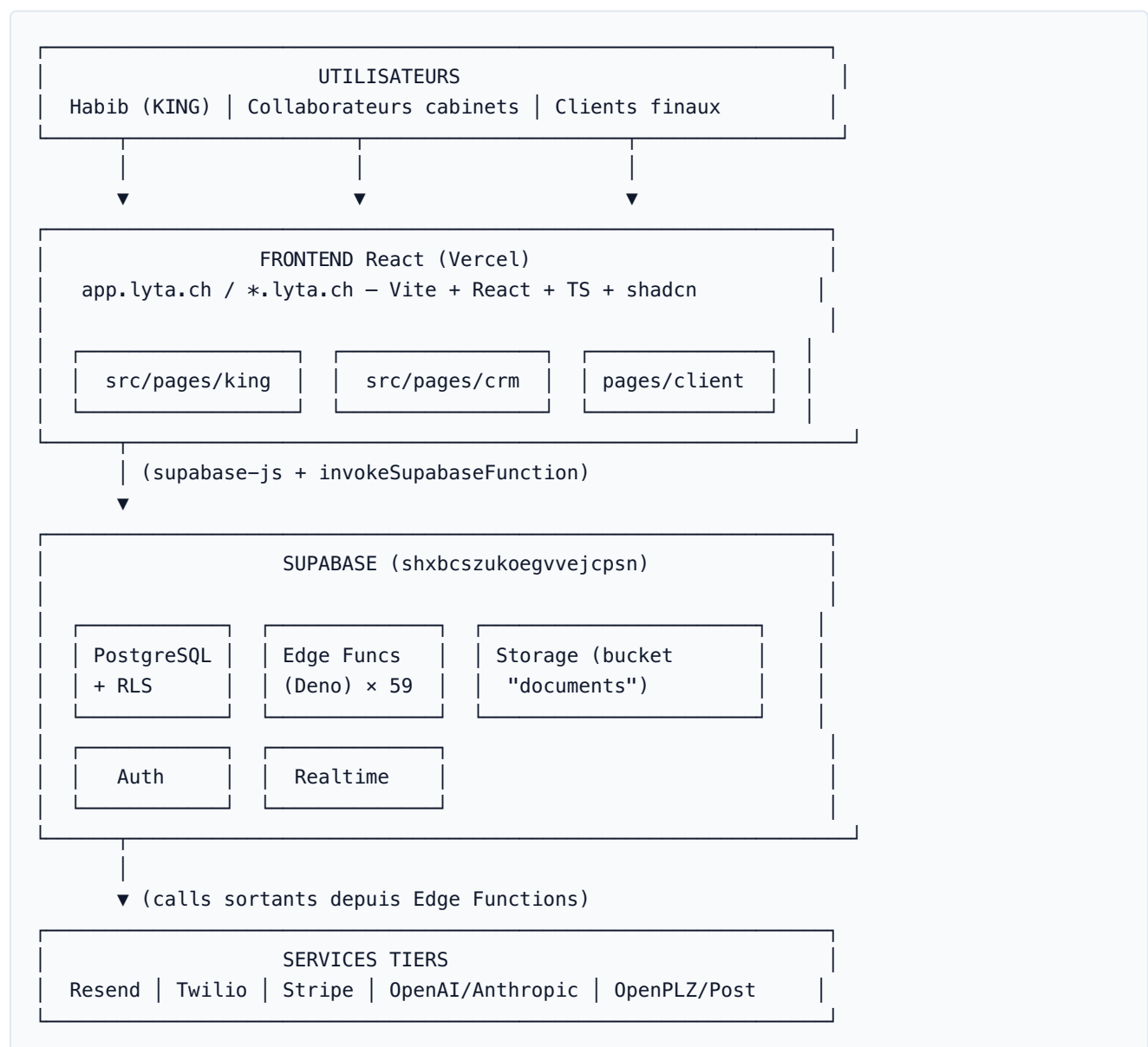
Tech	Version	Statut
Capacitor (@capacitor/core , android , ios)	8.0	Configuré (capacitor.config.ts à la racine) mais build natif non-distribué à date. L'app PWA web suffit pour le mobile en pratique.

2.5 Versions Node / Build

- **Node** : géré côté Vercel (LTS 20.x)
- **package manager** : `npm` (lockfile `package-lock.json` versionné)
- **Build prod** : `npm run build` (Vite → `dist/`)
- **Dev local** : `npm run dev` (Vite sur `localhost:5173` par défaut)
- **Lint** : `npm run lint` (ESLint 9 + typescript-eslint 8 + react-hooks plugin)

3. Architecture globale

3.1 Vue macro



3.2 Structure des sources frontend

```

src/
├── pages/                # Routes top-level (15+ CRM, 19 KING, 9 client, 7 racines)
│   ├── crm/              # /crm/* – espace courtier
│   ├── king/             # /king/* – espace admin plateforme (Habib)
│   ├── client/           # /client/* – espace client final
│   ├── Connexion.tsx     # /connexion
│   ├── Signer.tsx        # /signer/:token – flow signature à distance
│   ├── DeposerContrat.tsx
│   ├── ResetPassword.tsx
│   ├── FinaliserInscription.tsx
│   └── NotFound.tsx
├── components/           # Composants réutilisables (~250+)
│   ├── ui/               # shadcn primitives (57 fichiers)
│   ├── crm/              # Composants spécifiques CRM (35 fichiers)
│   ├── crm/emails/       # Sous-modules : EmailHistory, etc.
│   ├── crm/clients/      # BulkAssignDialog, QuickContactDialog
│   ├── crm/publicite/    # EmailDeliveryHistory
│   ├── king/             # Composants KING (16 fichiers)
│   ├── signatures/       # MandatTemplate, PdfZonePicker, etc. (5 fichiers)
│   ├── client/           # ClaimForm, ClientNotificationBell, MobileBottomNav
│   ├── auth/             # SmsVerificationDialog
│   ├── quotas/           # TenantQuotaWidget
│   └── support/          # TenantSupportTickets
├── hooks/                # 50 hooks personnalisés (logique métier réutilisable)
│   ├── useAuth.tsx
│   ├── useUserTenant.tsx
│   ├── useClients.tsx
│   ├── usePolicies.tsx
│   ├── useCommissions.tsx
│   ├── usePermissions.tsx # Calcul des droits selon rôle + scope
│   ├── useTenantConsumption.tsx # Quotas en temps réel
│   ├── useMandatDispatch.tsx
│   └── ... (voir Annexe 13.1 pour liste complète)
├── lib/                  # Utilitaires partagés
│   ├── edgeFunctions.ts  # wrapper invokeSupabaseFunction (gère erreurs uniformément)
│   ├── tenantUrls.ts     # Calcul sous-domaine selon tenant
│   ├── ibanUtils.ts      # Validation IBAN suisse
│   ├── documentUpload.ts # Upload Supabase Storage avec compression
│   ├── audit.ts          # Log des actions sensibles
│   ├── policiesApi.ts    # Helpers polices/contrats
│   ├── clientNotifications.ts
│   ├── insuranceCompanyLogos.ts
│   ├── errorTranslations.ts
│   ├── sessionEnforcerStorage.ts
│   └── utils.ts          # cn(), classNames merger Tailwind
├── integrations/
│   └── supabase/
│       ├── client.ts     # Client supabase-js
│       └── types.ts      # Types auto-générés (Database, Tables, etc.)

```

```

├─ App.tsx          # Provider tree + routing global
├─ main.tsx         # Entry point Vite

```

3.3 Structure backend

```

supabase/
├─ config.toml      # Config Supabase CLI (functions verify_jwt, auth settings)
├─ migrations/      # 256 migrations SQL versionnées (timestamps croissants)
│  └─ *.sql
├─ functions/       # 59 edge functions Deno
│  └─ _shared/      # cors.ts, logger.ts, requireAuth.ts, etc.
│     └─ send-signature-invite/
│     └─ proxy-signature-pdf/
│     └─ dispatch-mandat-to-companies/
│     └─ create-client/
│     └─ ... (voir section 7 pour catalogue)

```

3.4 Flux d'authentification (vue macro)

1. Utilisateur → /connexion → email + mot de passe
2. Frontend appelle `supabase.auth.signInWithPassword`
3. Supabase Auth retourne `{ session, user }`
4. `session.access_token` (JWT) stocké en `localStorage` par `supabase-js`
5. Tous les appels `supabase.from('xxx')` incluent automatiquement le JWT
6. PostgREST decode le JWT → applique RLS basées sur `auth.uid()`
7. RLS policies consultent `user_tenant_roles` pour scoper l'accès
8. Si tenant mismatch → 0 row retourné (jamais d'erreur 403 – silent filter)

Détail dans la section [8. Auth & Permissions](#).

3.5 Pattern de routage tenant

LYTA utilise un **routing par sous-domaine** :

URL	Comportement
<code>app.lyta.ch</code>	Landing + connexion KING
<code>app.lyta.ch/king/*</code>	Espace KING (admin plateforme)
<code>advisy.lyta.ch</code>	Auto-redirection vers la connexion du tenant <code>advisy</code>
<code>advisy.lyta.ch/crm/*</code>	Espace courtier du tenant <code>advisy</code>
<code>advisy.lyta.ch/client/*</code>	Espace client final (côté tenant)
<code>advisy.lyta.ch/signer/:token</code>	Page de signature publique (token uniquement, pas d'auth)

Le sous-domaine est détecté côté frontend (`src/lib/tenantUrls.ts`) puis utilisé pour : 1. Résoudre le `tenant_id` via la table `tenants` (colonne `slug`) 2. Charger le branding (`tenant_branding` — logo, couleurs) 3. Filtrer toutes les queries Supabase via les RLS (qui s'appuient sur `user_tenant_roles`)

Important sécurité : le sous-domaine **n'est PAS** la source de vérité du filtrage tenant — c'est juste l'UI. La vraie isolation est en RLS Postgres. Voir [section 8](#).

4. Modèle de données

4.1 Tables principales (90+ au total)

Listées par domaine fonctionnel. Pour chaque table : rôle, RLS scoping, points d'attention.

4.1.1 Plateforme & Tenants

Table	Rôle
<code>tenants</code>	Cabinets utilisateurs de LYTA (1 ligne par client SaaS). Colonnes clés : <code>id</code> , <code>slug</code> , <code>name</code> , <code>status</code> , <code>plan_id</code> , <code>stripe_customer_id</code>
<code>tenant_branding</code>	Logo, couleurs primaires/secondaires par tenant (sous-domaine)
<code>tenant_branches</code>	Agences/succursales d'un même tenant
<code>tenant_app_settings</code>	Settings configurables par tenant (préférences, intégrations)
<code>tenant_email_automation</code>	Configuration emails automatiques (anniversaires, échéances)
<code>tenant_email_log</code>	Historique unifié des emails sortants (transactionnels + dispatch + campagnes). Vue lue par <code>EmailDeliveryHistory.tsx</code> .
<code>tenant_feature_flags</code>	Flags par tenant pour activer des features en beta
<code>tenant_security_settings</code>	Config 2FA, durée sessions, IP allow-list, etc.
<code>tenant_consumption</code>	Métriques d'usage par tenant (pour facturation overage)
<code>tenant_overage_events</code>	Dépassements de quotas facturés
<code>tenant_quota_alerts</code>	Notifications de seuils atteints
<code>tenant_limits</code>	Limites par tenant (override des limites du plan)
<code>tenant_limits_audit</code>	Historique des modifications de limites
<code>tenant_billable_services</code>	Services additionnels facturables (ex: scans IA en sus)
<code>tenant_document_types</code>	Types de docs customisables par tenant
<code>tenant_product_commission_overrides</code>	Overrides de tarification commissions par produit/tenant
<code>tenant_roles</code>	Rôles personnalisés par tenant (en plus de admin/manager/agent)
<code>tenant_role_permissions</code>	Permissions granulaires associées aux rôles
<code>platform_plans</code>	Plans SaaS disponibles (Starter, Pro, Enterprise)
<code>platform_modules</code>	Modules activables (CRM Clients, Signatures, Smartflow...)
<code>plan_modules</code>	Mapping plan ? modules inclus
<code>plan_quotas</code>	Quotas par plan (max clients, max scans/mois, etc.)
<code>platform_usage_logs</code>	Audit cross-tenant des actions critiques (vu par KING)
<code>platform_settings</code>	Settings globaux de la plateforme
<code>pending_signups</code>	

Table	Rôle
	File d'attente des demandes d'inscription (avant validation par KING)

4.1.2 Utilisateurs & Permissions

Table	Rôle
<code>profiles</code>	Profil utilisateur (id = <code>auth.users.id</code> , prénom, nom, photo, langue)
<code>user_roles</code>	Rôle global d'un user (ex: <code>king</code>) — legacy, à vérifier vs <code>user_tenant_roles</code>
<code>user_tenant_roles</code>	Rôle d'un user dans un tenant (<code>admin</code> , <code>manager</code> , <code>agent</code>) + scope (<code>global</code> , <code>team</code> , <code>personal</code>)
<code>user_tenant_assignments</code>	Assignments multi-tenants si un user appartient à plusieurs cabinets
<code>user_app_connections</code>	Connexions externes (ex: API tierces liées au compte user)
<code>collaborator_permissions</code>	Permissions granulaires par collaborateur (override des permissions du rôle)

4.1.3 CRM Clients

Table	Rôle
<code>clients</code>	Le carnet de clients du cabinet (personnes physiques). Colonnes clés : <code>id</code> , <code>tenant_id</code> , <code>assigned_agent_id</code> , <code>first_name</code> , <code>last_name</code> , <code>email</code> , <code>phone</code> , <code>birth_date</code> , etc.
<code>family_members</code>	Membres du foyer rattachés à un client (conjoint, enfants). Peuvent eux-mêmes être des clients du cabinet.
<code>clients_safe</code>	Vue PostgreSQL retournant <code>clients</code> avec certaines colonnes masquées selon le scope (legacy ?)
<code>documents</code>	Documents attachés à un client (PDFs uploadés, scans, polices, mandats signés)
<code>document_categories</code>	Catégories de docs (Identité, Police, Décompte, Mandat...)
<code>document_templates</code>	Templates de docs pré-remplis utilisables par le cabinet
<code>document_reminders</code>	Rappels d'échéances liés aux documents (renouvellement carte ID, etc.)
<code>documents_expiring_soon</code>	Vue PostgreSQL listant les docs à échéance dans les 60 jours

4.1.4 Contrats / Polices

Table	Rôle
<code>policies</code>	Polices d'assurance d'un client (vie, LAMal, RC, etc.). Liée à <code>clients</code> + <code>insurance_companies</code> + <code>insurance_products</code>
<code>contracts</code>	Table legacy ? À vérifier — possible alias historique de <code>policies</code>
<code>claims</code>	Sinistres déclarés sur une police
<code>claim_documents</code>	Documents joints à un sinistre

4.1.5 Compagnies & Produits d'assurance

Table	Rôle
<code>insurance_companies</code>	Catalogue des compagnies (AXA, Helsana, Swiss Life...) — gérées par KING
<code>insurance_products</code>	Catalogue des produits par compagnie (LAMal Basis, LCA, 3a, 3b, LPP...)
<code>company_contacts</code>	Contacts dans une compagnie (email général, email mandat, téléphone) — utilisé par le dispatch mandat

4.1.6 Commissions & Comptabilité

Table	Rôle
<code>commissions</code>	Commissions calculées sur les polices
<code>commission_rules</code>	Règles tarifaires par compagnie/produit (taux, plafonds)
<code>commission_tiers</code>	Paliers de tiering (volume → taux variable)
<code>commission_history</code>	Historique des modifications de règles
<code>commission_statements</code>	Décomptes mensuels/trimestriels reçus des compagnies
<code>commission_statement_lines</code>	Lignes individuelles des décomptes (après import OCR)
<code>retrocommissions</code>	Rétrocessions sur agents/sous-agents
<code>decomptes</code>	Décomptes payés aux collaborateurs (paie)
<code>decompte_lines</code>	Lignes de décomptes par collaborateur
<code>payouts</code>	Paiements aux agents externes ou apporteurs
<code>qr_invoices</code>	Factures QR suisses émises (clients ou collaborateurs)
<code>qr_invoice_logs</code>	Audit des QR-factures émises
<code>transactions</code>	Mouvements comptables génériques
<code>reserve_accounts</code>	Comptes de réserve (provisions retours commissions)
<code>reserve_transactions</code>	Mouvements sur les comptes de réserve
<code>invoice_items</code>	Items facturables (jamais nullable, business logic)

4.1.7 Signatures & Mandats

Table	Rôle
<code>signature_requests</code>	Demandes de signature (mandat de gestion + docs importés). Colonnes clés : <code>access_token</code> (UUID public), <code>document_kind</code> (<code>mandat_gestion</code> , <code>imported</code> , <code>autre</code>), <code>status</code> (<code>draft</code> , <code>sent</code> , <code>viewed</code> , <code>signed</code> , <code>cancelled</code> , <code>refused</code>), <code>signature_zone</code> (jsonb coords normalisées), <code>preview_file_key</code> , <code>signed_file_key</code>
<code>mandat_dispatch_log</code>	Log par compagnie du dispatch d'un mandat signé (status, sent_at, erreur Resend)

4.1.8 Communications

Table	Rôle
<code>tenant_email_log</code>	Source de vérité historique emails sortants (kind = <code>signature_invite</code> , <code>mandat_signed</code> , <code>mandat_dispatch</code> , <code>account_created</code> , <code>campaign</code> , <code>crm_email</code> , <code>lpp_search</code> , <code>transactional</code> ...)
<code>scheduled_emails</code>	Emails programmés à envoyer (campagnes, anniversaires)
<code>email_templates</code>	Templates emails customisables par tenant
<code>sms_verifications</code>	Codes SMS Twilio en attente de vérification
<code>notifications</code>	Notifications in-app pour collaborateurs CRM
<code>messages</code>	Messages internes (chat ?) — à vérifier
<code>king_notifications</code>	Notifications pour Habib (alertes plateforme)

4.1.9 Smartflow / Scans

Table	Rôle
<code>document_scans</code>	Scans uploadés en attente de classification
<code>document_scan_results</code>	Résultats OCR + IA (champs extraits)
<code>document_scan_audit</code>	Audit des actions sur les scans
<code>scan_batches</code>	Lots de scans (traitement en bulk)
<code>lpp_search_requests</code>	Recherches LPP (2e pilier) groupées

4.1.10 Workflows & IA

Table	Rôle
<code>workflow_definitions</code>	Définitions de workflows automatisés (triggers + actions)
<code>workflow_executions</code>	Logs d'exécutions des workflows
<code>ai_conversations</code>	Conversations avec assistant IA
<code>ai_messages</code>	Messages individuels (rôle, content, tokens)
<code>ai_leads</code>	Leads générés par l'IA (prospects qualifiés)
<code>ai_rate_limits</code>	Quotas IA par tenant/user

4.1.11 Affiliés & Partenaires

Table	Rôle
<code>affiliates</code>	Programme d'affiliation (apporteurs LYTA)
<code>affiliate_commissions</code>	Commissions versées aux affiliés
<code>partners</code>	Partenaires commerciaux (cabinets référencés)

4.1.12 Support & Audit

Table	Rôle
<code>support_tickets</code>	Tickets support entre tenants et Habib
<code>support_ticket_messages</code>	Messages d'un ticket
<code>audit_logs</code>	Audit des actions sensibles (modifs critiques)
<code>king_audit_log</code> / <code>king_audit_logs</code>	Audit KING (double table — legacy à consolider)
<code>app_usage_logs</code>	Logs d'usage de l'app par les utilisateurs
<code>webhooks</code>	Webhooks sortants configurés par tenant
<code>webhook_logs</code>	Logs des webhooks émis
<code>api_rate_limits</code>	Rate limits par API endpoint
<code>external_apps</code>	Apps externes connectées (intégrations type Zapier custom)

4.2 Conventions de nommage

- **Tables** : `snake_case` pluriel (`clients` , `signature_requests`)
- **Colonnes** : `snake_case` (`tenant_id` , `created_at`)
- **Foreign keys** : `<entity>_id` (`tenant_id` , `client_id` , `created_by`)
- **Timestamps** : `created_at` , `updated_at` (toujours `timestampz`)
- **Soft delete** : pas systématique — vérifier au cas par cas (`deleted_at` quand utilisé)
- **JSONB** : pour payloads flexibles (`payload` , `metadata` , `settings` , `signature_zone`)
- **Enums Postgres** : utilisés pour `status` , `document_kind` , `role` , etc. (voir migrations)

4.3 Triggers & fonctions critiques

- `set_updated_at()` — trigger BEFORE UPDATE qui maintient `updated_at`
- `set_tenant_id_from_jwt()` — trigger BEFORE INSERT qui force `tenant_id` depuis le JWT (anti-injection)
- `can_access_client(client_uuid)` — fonction SECURITY DEFINER appelée par les RLS
- `get_user_tenant_id()` — retourne le tenant actif de l'utilisateur courant
- `my_collab_id_for_active_tenant()` — retourne l'ID collaborateur dans le tenant actif

- `_client_is_in_team(client_uuid, manager_uuid)` — vérifie si un client appartient à l'équipe d'un manager

Pour l'audit complet des RLS, voir le **Doc 2 confidentiel** (Security Audit).

5. Multi-tenancy & permissions

5.1 Le principe

LYTA est **multi-tenant strict** : chaque cabinet vit dans le même cluster Postgres, isolé par RLS. **Aucune donnée ne doit jamais fuiter d'un tenant à un autre**, sauf via les outils KING explicitement prévus pour ça.

L'isolation repose sur **trois couches** :

1. **Sous-domaine** (UI uniquement, non-sécurisé) — détermine le branding et le tenant à charger
2. **JWT Supabase** (auth) — contient l'`auth.uid()` de l'utilisateur connecté
3. **RLS Postgres** (sécurité réelle) — filtre toutes les queries selon `user_tenant_roles`

5.2 Rôles et scopes

Rôles de base (table `user_tenant_roles.role`)

Rôle	Description	Voit
<code>admin</code>	Administrateur du tenant	Tout le tenant
<code>manager</code>	Manager d'équipe	Lui-même + les agents qu'il supervise + leurs clients
<code>agent</code>	Agent commercial	Lui-même + ses clients assignés
<code>king</code>	Habib / Optimislink	Tous les tenants (read), opérations admin (write)

Scopes (table `user_tenant_roles.scope`)

Le scope module la visibilité au sein du rôle :

Scope	Effet
<code>global</code>	Voit toutes les données du tenant (typique pour admin)
<code>team</code>	Voit son équipe + ses subordonnés (typique pour manager)
<code>personal</code>	Voit uniquement ses propres données (typique pour agent)

Exemple concret : Stéphane (agent JCG) a `role = 'agent'` et `scope = 'personal'`. Il ne voit que ses clients assignés (`clients.assigned_agent_id = collab_id_de_stephane`). Voir l'incident timeout RLS résolu dans la session du 3 juin (`useClients.tsx` applique un filtre `or(assigned_agent_id.eq.<myCollabId>, id.eq.<myCollabId>)` côté front pour éviter la lenteur des RLS scope-aware en SQL).

5.3 Fonctions de scope (`useUserTenant`, `usePermissions`)

Le hook `useUserTenant()` (frontend) résout : - `tenantId` (UUID du tenant actif) - `myScope` (`'global'` | `'team'` | `'personal'`) - `myCollabId` (UUID collaborateur dans ce tenant) - `role`

Le hook `usePermissions()` calcule des **droits dérivés** (peut-il créer un client ? voir les commissions ? exporter ?) en fonction du rôle + scope + collaborator_permissions.

5.4 Routage et résolution tenant côté frontend

```
// Pseudo-code de résolution du tenant actif
1. URL = "advisy.lyta.ch" → slug = "advisy"
2. supabase.from('tenants').select('id').eq('slug', 'advisy').single()
3. tenant.id stocké dans le contexte UserTenantContext
4. Toutes les queries qui ont besoin du tenant utilisent useUserTenant().tenantId
```

5.5 Multi-cabinets (un user dans plusieurs tenants)

Un même utilisateur peut être collaborateur dans plusieurs cabinets (table `user_tenant_assignments`). Dans ce cas : - Switch tenant via UI (sélecteur en haut du CRM) - Le tenant actif est stocké en localStorage + cookie (à vérifier dans `lib/tenantUrls.ts`) - Les RLS s'appuient sur le tenant actif retourné par `get_user_tenant_id()`

5.6 Cas KING (Optimislink / Habib)

Habib (rôle `king`) a un accès cross-tenant pour les opérations plateforme : - Lister/voir tous les tenants - Activer/désactiver un tenant - Voir les métriques d'usage - Impersonate un tenant (edge function `king-impersonate-tenant` — à auditer pour traçabilité) - Modifier les plans, quotas, facturation

Toutes les actions KING sont auditées dans `king_audit_logs` (et `king_audit_log` — doublon legacy à consolider).

5.7 Espace client final

Les clients finaux (table `clients`) **n'ont pas** d'authentification Supabase classique. Ils accèdent à leur espace via : - **Magic link email** (provision via `send-client-message` ?) - Token de session court (à confirmer dans `useAuth`)

Leur visibilité : uniquement leurs propres données (dossier, polices, sinistres, messages avec leur conseiller).

À auditer : confirmer comment exactement le client final est authentifié et quelle RLS s'applique à lui (différente de celle des collaborateurs).

6. Modules métier

 **Convention de cette section** : chaque module suit la même structure (rôle métier, surface code, edge functions, business rules clés, gotchas, RLS critique). Les **modules 1-6** sont documentés ci-dessous (passe 2). Les modules 7-18 viennent dans les passes suivantes.

6.1 Module 1 — Auth & Onboarding

6.1.1 Rôle métier

Gérer **trois flows distincts** d'arrivée d'un utilisateur sur LYTA :

1. **Self-signup post-paiement Stripe** : un nouveau cabinet paie via le site marketing (`lyta.ch/access`, hébergé sur Lovable), puis est redirigé sur LYTA pour finaliser → création tenant + admin + DNS + Vercel + Resend en chaîne.
2. **Connexion existante** : email + password, avec **vérification SMS 2FA optionnelle** (si activée au niveau du tenant) + **check HavelBeenPwned** sur les nouveaux mots de passe.
3. **Invitation collaborateur** : un admin/manager invite un collègue → email avec magic link → `FinaliserInscription.tsx`.

6.1.2 Surface code

Fichier	Rôle
<code>src/hooks/useAuth.tsx</code> (~700 LOC)	Hook central : signIn, signUp, signOut, resetPassword, MFA SMS challenge, session persistence (sessionStorage), mock SMS client pour la phase challenge
<code>src/pages/Connexion.tsx</code>	Formulaire login + intégration <code>SmsVerificationDialog</code>
<code>src/pages/FinaliserInscription.tsx</code>	Page de finalisation invitation (set password + complete profile)
<code>src/pages/ResetPassword.tsx</code>	Reset password via magic link Supabase
<code>src/components/auth/SmsVerificationDialog.tsx</code>	UI saisie code SMS Twilio Verify
<code>src/hooks/usePasswordCheck.tsx</code>	Validation password (longueur + caractères + HIBP)
<code>src/hooks/useSessionTimeout.ts</code>	Auto-logout après inactivité (configurable par tenant via <code>tenant_security_settings</code>)
<code>src/hooks/useForcedLogoutAfter.ts</code>	Logout forcé après changement critique (rôle modifié, MFA enrôlée...)
<code>src/lib/sessionEnforcerStorage.ts</code>	Helpers persistence session

6.1.3 Edge Functions liées

Function	Rôle	Auth
<code>provision-self-signup-tenant</code>	Orchestrateur du flow Stripe → tenant + admin + DNS + Resend	Public (sécurité = secret session Stripe)
<code>tenant-onboarding</code>	Provisioning DNS Cloudflare + domaine custom Vercel + setup Resend	<code>verify_jwt = false</code>
<code>create-tenant-admin</code>	Crée le user admin + envoie magic link bienvenue (Resend)	<code>verify_jwt = false</code>
<code>create-user-account</code>	Crée un user (utilisé pour invitations collaborateur ou client)	<code>verify_jwt = false</code>
<code>create-collaborator</code>	Wrapper côté CRM pour création collaborateur + assignment rôle/scope	Authentifié
<code>send-verification-sms</code>	Envoie code SMS Twilio Verify	<code>verify_jwt = false</code>
<code>verify-sms-code</code>	Valide le code SMS et complète l'authentification	<code>verify_jwt = false</code>
<code>send-password-reset</code>	Reset password (Resend)	<code>verify_jwt = false</code>
<code>resend-signup-finalization</code>	Réémet l'email de finalisation si l'user n'a pas reçu	Authentifié
<code>delete-user-account</code>	Suppression user (RGPD)	KING-only

6.1.4 Business rules clés

- **Mot de passe** : minimum 8 caractères + **vérification HavelBeenPwned** via API publique (préfixe SHA-1, k-anonymity). Si le password apparaît dans une fuite connue → refus.
- **MFA SMS** : configurable par tenant dans `tenant_security_settings`. Si activé, après le `signInWithPassword` réussi, un challenge SMS est lancé sur le numéro stocké dans le profil. Tant que le code n'est pas validé, l'utilisateur reste dans un état "pendingSmsVerification" (stocké en `sessionStorage`).
- **Magic link expiry** : 24h (anciennement 1h, augmenté car les users ne cliquaient pas à temps — voir migration `20260603...` et `config.toml` `[auth.email].otp_expiry = 86400`).
- **OTP length** : 8 caractères (`otp_length = 8` dans `config.toml`).
- **Session timeout** : configurable par tenant. Auto-logout si l'user est inactif (mouvement souris/clavier surveillés via `useSessionTimeout`).
- **Self-signup idempotence** : si on relance `provision-self-signup-tenant` avec une `session_id` Stripe déjà traitée → retourne le tenant existant au lieu de le recréer.

6.1.5 Flow self-signup détaillé

1. Marketing site (Lovable) → POST /provision-self-signup-tenant
 { stripe_session_id, slug, tenant_name, admin_email, admin_first_name, ... }
2. Edge function:
 - a. stripe.checkout.sessions.retrieve(stripe_session_id) – vérifie payment_status === 'paid'
 - b. SELECT FROM tenants WHERE stripe_session_id = ? – idempotence
 - c. INSERT INTO tenants (status = 'pending_setup', plan_id, stripe_customer_id, ...)
 - d. Appelle create-tenant-admin → INSERT auth.users + INSERT user_tenant_roles + envoie magic link via Resend
 - e. Appelle tenant-onboarding step="full" → crée DNS Cloudflare CNAME + ajoute domaine custom à Vercel + crée audience Resend
 - f. INSERT king_notifications (Habib reçoit alerte "Nouveau tenant {slug}")
 - g. Retourne {tenant_id, slug, url: "https://{slug}.lyta.ch"}
3. User clique le magic link reçu → FinaliserInscription
 - a. Set password (validé HIBP + longueur)
 - b. Optionnel : complète le profil
 - c. Redirige vers https://{slug}.lyta.ch/crm

6.1.6 Gotchas

- **Double supabase client** : `useAuth` instancie un **second client** (`smsClient`) avec sa propre storage isolée pour la phase challenge SMS — sinon, après `signInWithPassword`, l'user serait déjà "connecté" du point de vue de Supabase, et un refresh page contournerait le SMS. Le challenge se résout en migrant explicitement la session du `smsClient` vers le `supabase` principal après validation.
- **Provisioning Vercel** : `tenant-onboarding` peut échouer si le quota Vercel domaines est atteint ou si la zone Cloudflare est mal configurée. Le tenant est laissé en `pending_setup` et Habib reçoit une notif. **Pas de rollback automatique.**
- **Reset password depuis sous-domaine tenant** : la redirection se fait via `window.location.origin` → atterrit bien sur le sous-domaine, mais l'URL doit être dans `additional_redirect_urls` du `config.toml` (déjà : `https://*.lyta.ch/**`).

6.1.7 RLS critique

Tables impliquées : `profiles`, `user_tenant_roles`, `user_tenant_assignments`, `tenants`, `pending_signups`, `sms_verifications`.

- **profiles** : RLS = `id = auth.uid()` (chaque user voit/édite uniquement son propre profil)
- **user_tenant_roles** : SELECT scopé au tenant actif, INSERT/UPDATE réservé aux admins via fonctions dédiées
- **tenants** : SELECT = membre du tenant ; INSERT/UPDATE = KING uniquement (sauf provisioning via `service_role`)

6.2 Module 2 — CRM Clients

6.2.1 Rôle métier

Carnet de **personnes physiques** clientes du cabinet. C’est le **module le plus utilisé** de LYTA — point d’entrée de tous les autres modules (polices, commissions, signatures, mandats, sinistres, documents).

Couvre : - Création / édition / archivage de clients - **Membres de famille** rattachés au foyer (conjoint, enfants — peuvent être eux-mêmes clients) - **Assignment** d’un client à un agent (`assigned_agent_id`) - **Bulk assign** (réassigner en masse N clients à un autre agent) - **Import prospects** (CSV/XLSX) - **Documents** liés au dossier client - **Suivis** (tasks/reminders sur un client) - **QuickContact** : popup compact “appeler / SMS / email” depuis n’importe quelle liste

6.2.2 Surface code

Fichier	Rôle
<code>src/pages/crm/clients/ClientsList.tsx</code>	Liste paginée des clients (50/page), filtres, recherche, BulkAssign
<code>src/pages/crm/clients/</code> (autres)	Pages détail / création / édition
<code>src/hooks/useClients.tsx</code> (~600 LOC)	Query principale + RLS scope-aware filter front + timeout 45s
<code>src/hooks/useFamilyMembers.tsx</code>	Gestion membres de famille
<code>src/hooks/useDocuments.tsx</code>	Documents par client
<code>src/components/crm/clients/BulkAssignDialog.tsx</code>	Dialog réassignation masse
<code>src/components/crm/clients/QuickContactDialog.tsx</code>	Popup contact rapide
<code>src/components/crm/clients/ImportFamilyMemberDialog.tsx</code>	Ajout membre famille (peut linker un client existant)
<code>src/components/crm/clients/ProspectImportDialog.tsx</code>	Import CSV/XLSX prospects

6.2.3 Edge Functions liées

Function	Rôle	Pourquoi elle existe
<code>create-client</code>	Fallback INSERT client via <code>service_role</code>	Bypass d'un bug RLS 42501 récurrent observé en prod sur Advisy et JCG. Le code source explique : “createClient front fait <code>.insert([...])</code> direct sur <code>public.clients</code> , PostgREST renvoie 403 / 42501 alors que toutes les conditions semblent vraies en tests SQL manuels” — mismatch SQL CLI vs PostgREST runtime. À investiguer en priorité par le dev (voir Dette technique §11).
<code>bypass-insert</code>	Fallback générique INSERT pour <code>family_members</code> + <code>documents</code>	Même raison que <code>create-client</code>

6.2.4 Business rules clés

- **Scope-aware visibility** : un agent ne voit que ses clients (`assigned_agent_id = my_collab_id`) — appliqué **côté frontend** dans `useClients.tsx` parce que les RLS scope-aware côté SQL faisaient timeout (Stéphane JCG : 80s). Code clé :

```
if (myCollabId && (myScope === "personal" || myScope === "team")) {
  query = query.or(`assigned_agent_id.eq.${myCollabId},id.eq.${myCollabId}`);
}
```

- **Timeout query** : `CLIENTS_QUERY_TIMEOUT_MS = 45_000` (bumpé depuis 12s parce que comptage + select sur 1000+ clients avec RLS lourdes prenait >12s).
- **Création client = peut créer aussi user portal** : si le client a un email, on crée optionnellement un `auth.users` pour qu'il accède à son espace.
- **Family members ? clients** : un `family_members` peut référencer un `clients.id` existant (linkage) ou être autonome (juste prénom/nom dans la table `family_members`). UX : “Importer un client existant comme membre de famille”.

6.2.5 Gotchas

- **Le bug RLS 42501 est toujours actif** côté Postgres — `create-client` n'est qu'un workaround. Tous les chemins de création client (formulaire CRM, ScanValidationDialog après Smartflow, ImportProspect, etc.) ont été migrés pour passer par `create-client` (commits `f25593f`, `79e2b75`).
- `clients_safe` : vue Postgres listée dans le schéma. Usage actuel **à confirmer** — probablement un legacy d'une époque où on retournait `clients` avec colonnes masquées selon le scope. **À auditer** : si plus utilisée, supprimer.
- `assigned_agent_id` référence un `tenant_users.id` ? Ou un `auth.users.id` ? À auditer dans la table `clients` schema.

6.2.6 RLS critique

- `clients` : `SELECT = tenant_id = get_user_tenant_id()` (RLS simple, depuis la révert anti-timeout du 3 juin). Le scoping personal/team est appliqué côté frontend.
- `family_members`, `documents` : `SELECT` via fonction `can_access_client(client_id)`.

6.3 Module 3 — CRM Polices / Contracts

6.3.1 Rôle métier

Gérer le **portefeuille d’assurances** d’un client : polices souscrites, échéances, primes, sinistres associés. C’est l’entité qui alimente les commissions et les dashboards de production.

6.3.2 Surface code

Fichier	Rôle
<code>src/pages/crm/CRMContracts.tsx</code>	Liste portefeuille (toutes polices du tenant)
<code>src/hooks/usePolicies.tsx</code>	Query principale paginée + helpers
<code>src/lib/policiesApi.ts</code>	Helpers <code>savePolicy()</code> (CREATE/UPDATE avec validation), normalisation produits
<code>src/hooks/usePolicies.tsx</code> (Policy type)	Type Policy complet (28+ champs)
<code>src/pages/DeposerContrat.tsx</code>	Page publique “dépôt de contrat” (clients peuvent déposer une police existante)
<code>src/pages/crm/CRMPropositions.tsx</code>	Propositions commerciales avant signature

6.3.3 Edge Functions liées

Function	Rôle
<code>save-policy</code>	INSERT/UPDATE police avec validation server-side (génère UUID, supprime <code>.select()</code> post-INSERT pour éviter RLS SELECT immédiat)
<code>deposit-contract</code>	Dépôt public d’un contrat par un client (depuis <code>DeposerContrat.tsx</code>)
<code>send-contract-deposit-email</code>	Notif au broker quand un client dépose un contrat

6.3.4 Business rules clés

- `policies` vs `contracts` : les **deux tables existent** en DB. Code et hooks utilisent `policies` (table actuelle). La table `contracts` semble être un **legacy résiduel** — d’ailleurs `save-policy/index.ts:196` fait référence à `module = "contracts"` qui est un **identifiant de module** dans `platform_modules` (le module s’appelle encore “contrats” UI-side, mais stocke ses données dans `policies`). **À auditer / nettoyer.**

- **Multi-produits par police** : une police peut couvrir plusieurs produits (LAMal + LCA + dentaire) → `products_data: Array<{productId, premium, ...}>` JSONB.
- **tenant_branch_id** : override par police de la branche du produit (utile quand le cabinet a plusieurs agences avec des tarifications différentes).
- **Statuts** : `draft`, `active`, `cancelled`, `expired`, etc. (à confirmer dans le schéma).
- **partner_id** : référence un partenaire externe (apporteur) qui a amené le contrat → utilisé pour les rétrocessions.

6.3.5 Gotchas

- **UUID backend forcé** : `save-policy` génère lui-même l'UUID au lieu de laisser Postgres le faire, pour pouvoir utiliser le même UUID dans des opérations downstream sans relire le row (`.select()` enlevé après INSERT/UPDATE car déclenche RLS SELECT qui peut bloquer — voir commit `d76c0a8`).
- **DeposerContrat public** : la page `/deposer-contrat` est accessible sans auth. Un client peut y déposer un contrat actuel — ça crée un `pending_signups` ou un dossier à traiter par le broker. Surface d'attaque à auditer.

6.4 Module 4 — CRM Commissions

6.4.1 Rôle métier

Le **module le plus complexe** de LYTA. Calcule, suit et reverse les commissions des compagnies vers le cabinet, puis du cabinet vers les agents (rétrocessions), avec gestion des **réserves**, **paliers** (tiering volume), **décomptes** mensuels.

Sous-domaines : - **Règles tarifaires** par compagnie/produit (taux, plafonds, paliers) - **Saisie manuelle** d'une commission ou **import** d'un décompte compagnie - **Smartflow Décomptes** : import + OCR + parsing IA du PDF de décompte compagnie (gpt-5) → lignes auto-matchées avec clients/polices → validation manuelle par le broker - **Rétrocessions** aux agents/sous-agents avec règles configurables - **Décomptes payés aux collaborateurs** (équivalent paie) - **Comptes de réserve** : retenue d'un % pour provisionner les retours/annulations futures

6.4.2 Surface code

Fichier	Rôle	Volume
<code>src/pages/crm/CRMCommissions.tsx</code>	Page principale liste + filtres + bannière “X commissions à valider”	1043 LOC
<code>src/components/crm/CommissionForm.tsx</code>	Formulaire commission (saisie manuelle ou pré-rempli depuis Smartflow)	836 LOC
<code>src/hooks/useCommissions.tsx</code>	Query + pagination + helpers	
<code>src/hooks/useCommissionParts.tsx</code>	Décomposition d’une commission en parts (agent / cabinet / réserve / TVA)	
<code>src/hooks/useCollaborateursCommission.tsx</code>	Vue par collaborateur (total par agent)	

6.4.3 Edge Functions liées

Function	Rôle
<code>scan-commission-statement</code>	Reçoit un <code>commission_statement_id</code> , lit le PDF de décompte, l’envoie à gpt-5 avec prompt structuré, parse les lignes, INSERT dans <code>commission_statement_lines</code> , et lance la RPC <code>match_commission_line</code> pour auto-match avec clients/polices existants

6.4.4 Business rules clés

- **Calcul** : `commission_amount = base_amount × rate / 100` (rate dépend de `commission_rules` ou `commission_tiers` ou `tenant_product_commission_overrides`)
- **Réserve** : `reserve_amount = commission_amount × reserve_rate / 100`. Le reste (`net`) part vers la rétrocession agent.
- **Réserves par collaborateur** : `collaborateurs.reserve_rate` (configurable par admin). Voir `CRMCompta.tsx:229–244`.
- **Matching auto Smartflow** : `match_commission_line` essaie de matcher par numéro de police, puis par nom/prénom client. Si match → la ligne est pré-validée. Sinon → broker valide manuellement.
- **Validation broker** : chaque ligne d’un décompte doit être validée individuellement avant d’être comptabilisée dans les KPIs. Bannière “X commissions à valider” reste visible jusqu’à clean-up.
- **Tiering** : `commission_tiers` permet des taux progressifs (ex: 0-50 polices = 10%, 51-100 = 12%, 101+ = 15%).

6.4.5 Gotchas

- `commission_statement_lines` n’est pas dans les types Supabase auto-générés : erreur tsc visible `Argument of type '"commission_statement_lines"' is not assignable to parameter of type '[list of tables]'` dans `CommissionForm.tsx:387`. **Type-gen Supabase à régénérer.**

- `scan-commission-statement` utilise `gpt-5` : modèle confirmé dans le commentaire du fichier. Coût par scan à monitorer (un décompte mensuel = 50-200 lignes).
- **Match ambigu** : si plusieurs polices du client peuvent matcher la ligne (homonymes, polices multiples), la RPC choisit la plus récente — peut donner des erreurs silencieuses. **À auditer.**

6.5 Module 5 — CRM Compagnies d'assurance

6.5.1 Rôle métier

Catalogue centralisé des **compagnies** (AXA, Helsana, Swiss Life...) et leurs **produits** (LAMal Basis, LCA, 3a...). Géré principalement par **KING** (catalogue partagé entre tenants) avec possibilité de produits/contacts propres au tenant.

Utilisé pour : - Création d'une police (sélection compagnie + produit) - Dispatch mandat (envoi du PDF signé aux contacts mandat de chaque compagnie) - Calcul commissions (compagnie → règles de commission) - Affichage des **logos** des compagnies dans toute l'UI

6.5.2 Surface code

Fichier	Rôle
<code>src/pages/crm/CRMCompagnies.tsx</code>	Liste compagnies + produits (1158 LOC) — vue cabinet
<code>src/components/crm/CompanyContactsPanel.tsx</code>	Gestion contacts par compagnie (email mandat, email général, téléphone)
<code>src/hooks/useInsuranceCompanies.tsx</code>	Query compagnies (du catalogue)
<code>src/hooks/useInsuranceProducts.tsx</code>	Query produits
<code>src/hooks/useCompanyContacts.tsx</code>	Query contacts
<code>src/components/crm/InsuranceCompanyLogo.tsx</code>	Composant logo (CDN local via <code>insuranceCompanyLogos.ts</code>)
<code>src/lib/insuranceCompanyLogos.ts</code>	Mapping nom compagnie → URL logo local
<code>src/pages/king/KingCatalog.tsx</code>	Gestion du catalogue par KING (cross-tenant)
<code>src/components/king/CompanyCatalogManager.tsx</code>	UI KING pour CRUD compagnies/produits

6.5.3 Business rules clés

- **Catalogue cross-tenant** : `insurance_companies` + `insurance_products` sont **partagés** entre tous les tenants (gérés par KING).
- **Produits tenant-spécifiques** : un tenant peut ajouter des produits qui n'existent pas dans le catalogue global (`insurance_products.tenant_id IS NOT NULL`).
- **Contacts par compagnie** : `company_contacts` stocke un `contact_type` (general / mandat / sinistre) + channel (email / phone). Le **dispatch mandat** utilise `contact_type = 'mandat'` en priorité.
- **Logos** : servis depuis `/public/insurance-logos/` (assets locaux). Fallback : avatar avec initiales.

6.5.4 Gotchas

- **Aucune mention de “mandat” dans `CRMCompagnies.tsx`** (vérifié dans 1158 LOC) → la page **n’affiche pas le statut de dispatch des mandats**. C’est volontaire — le suivi se fait via `tenant_email_log` dans CRM Publicité. **À documenter pour le dev** : ne pas tenter de l’ajouter sans aligner avec Habib.

6.6 Module 6 — CRM Compta

6.6.1 Rôle métier

Vue **comptable** côté cabinet : décomptes payés aux collaborateurs, comptes de réserve, factures QR suisses émises/reçues, transactions, exports.

6.6.2 Surface code

Fichier	Rôle	Volume
<code>src/pages/crm/CRMCompta.tsx</code>	Page principale (multi-onglets : décomptes, réserves, QR-factures, exports)	1205 LOC
<code>src/hooks/useQRInvoices.tsx</code>	QR-factures suisses	
<code>src/hooks/useCollaborateursCommission.tsx</code>	Synthèse par collaborateur	

6.6.3 Tables clés

- `decomptes` + `decompte_lines` : décomptes périodiques par collaborateur
- `qr_invoices` + `qr_invoice_logs` : QR-factures suisses émises (générées via `qrcode` lib)
- `transactions` : mouvements génériques
- `reserve_accounts` + `reserve_transactions` : comptes de réserve par collaborateur
- `payouts` : paiements externes (apporteurs, partenaires)

6.6.4 Business rules clés

- **Réserves** : à chaque commission encaissée, un % (`reserve_rate`) est prélevé pour le compte de réserve du collaborateur. Sert à provisionner les retours / annulations de polices (la compagnie peut reprendre la commission jusqu’à N mois après).
- **Calcul net agent** : `net = commission_brut - reserve_amount - tva (si applicable)`. Voir `CRMCompta.tsx:241-244`.
- **QR-facture suisse** : génère le code QR conforme à la norme ISO 20022 (IID, IBAN QR-only, montant CHF). Lib `qrcode` + payload structuré.
- **Module conditionnel** : `hasQRInvoiceAccess = hasModule('qr_invoice')` — disponible uniquement si le plan du tenant inclut le module QR-facture (`plan_modules`).

6.6.5 Gotchas

- **Précision financière** : tous les montants sont en **NUMERIC(12,2)** côté Postgres. **Ne JAMAIS** utiliser de FLOAT côté JS — toujours `Number(value).toFixed(2)` avant d’afficher, ou utiliser une lib type `dinero.js` pour les calculs sensibles.
- **Pas d’audit trail systématique** sur `transactions` / `decompte_lines` — un INSERT ou DELETE manuel peut altérer les chiffres sans trace. **À renforcer**.

6.7 Module 7 — CRM Smartflow / Scan IA / Propositions

6.7.1 Rôle métier

Module IA qui **automatise la saisie** de documents reçus par les courtiers : - **Scan d’un PDF/photo** (police existante, pièce d’identité, ancien décompte...) - **OCR + classification IA** (gpt-5 / Anthropic) en 1 des ~10 catégories métier - **Extraction de champs structurés** (numéro de police, dates, montants, compagnie, produit) - **Validation manuelle** par le broker dans `ScanValidationDialog` - **Création automatique** du client / police / commission / family member liés

Sous-module **Propositions** : devis commerciaux générés à partir d’un dossier client (avant signature de mandat).

6.7.2 Surface code

Fichier	Rôle
<code>src/pages/crm/CRMPropositions.tsx</code>	Page principale Smartflow (onglets Scans / Batches)
<code>src/components/crm/ia-scan/</code>	Sous-dossier : <code>ScanBatchUpload</code> , <code>ScanBatchReview</code> , <code>ScanValidationDialog</code>
<code>src/hooks/usePendingScans.tsx</code>	Scans en attente de validation
<code>src/hooks/useScanBatches.tsx</code>	Lots de scans (traitement bulk)
<code>src/hooks/usePendingProducts.tsx</code>	Produits détectés mais pas encore dans le catalogue → file d’attente KING

6.7.3 Edge Functions liées

Function	Rôle
<code>scan-document</code>	Scan unitaire : OCR (Mistral OCR ou équivalent) + classification + extraction champs
<code>classify-batch-documents</code>	Traitement en lot avec quotas (<code>reserveTenantQuota</code> / <code>releaseTenantQuota</code>)
<code>scan-commission-statement</code>	Cas particulier : décomptes compagnies (voir Module 4)
<code>_shared/ai.ts</code>	Helpers communs IA (<code>fetchAiChatCompletions</code> , <code>getAiModel</code> , <code>buildAiError</code> , <code>isAiTimeoutError</code>)

6.7.4 Catégories de classification

D'après `classify-batch-documents/DOC_CLASSIFICATIONS` : - `identity_doc` : pièce d'identité - `old_policy` : police actuelle (à résilier) - *(liste complète à compléter par lecture du fichier)*

Selon la catégorie, des **enrichissements automatiques** s'appliquent : - Lemanica → couvre VIE (3e pilier) ET LPP (2e pilier) selon le contenu réel du doc (fix commit `8853c94`) - Match VIE/LPP avec compagnies via fuzzy matching

6.7.5 Business rules clés

- **Quotas par tenant** : chaque scan IA décrémente le quota `scans_per_month` du plan. Si dépassement → bloque ou facture overage selon `apply-monthly-override`.
- **Création client auto** : après validation, si le client n'existe pas → INSERT via `bypass-insert` (workaround RLS).
- **Pending products** : si l'IA détecte un produit qui n'est pas dans le catalogue → envoyé en file d'attente KING pour validation par Habib.

6.7.6 Gotchas

- **Bug RLS auquel Smartflow est confronté** : tous les INSERT de `family_members` et `documents` issus du scan passent par `bypass-insert`, sinon même 42501 que création client manuelle.
- **gpt-5 coûteux** : 50-200 lignes par décompte × N décomptes/mois × N tenants. Surveiller la facturation OpenAI / Anthropic.
- **OCR sur images de mauvaise qualité** : taux d'erreur élevé sur photos prises avec smartphone mal éclairé. Pas de retry user-friendly.

6.8 Module 8 — CRM Signatures (mandat + imported + dispatch)

6.8.1 Rôle métier

Le module sur lequel on a le plus travaillé en juin 2026. Permet au courtier d'envoyer un document pour signature électronique à distance à un client.

Deux flows distincts : 1. **Mandat de gestion** : doc juridique généré depuis un template React (`MandatTemplate`), avec champs dynamiques (cabinet, client, branche). Zone de signature historiquement en bas du paragraphe d'engagement. 2. **Document importé** : PDF arbitraire uploadé par le broker (procuration, lettre résiliation, etc.). Le **signataire** drague la zone de signature où il veut (depuis juin 2026).

Post-signature mandat : dispatch automatique aux compagnies via `MandatDispatchPanel` (bouton manuel).

6.8.2 Surface code

Fichier	Rôle
<code>src/pages/crm/CRMSignatures.tsx</code>	Page liste des demandes (en cours / signées / refusées)
<code>src/pages/Signer.tsx</code>	Page publique de signature (URL <code>/signer/:token</code> , pas d'auth, vérif par token UUID)
<code>src/components/signatures/PendingSignaturesPanel.tsx</code>	Panneau dans la fiche client listant signatures en cours/passées
<code>src/components/signatures/MandatTemplate.tsx</code>	Template React du mandat (HTML rendu en PDF via html2canvas + pdf-lib)
<code>src/components/signatures/MandatDispatchPanel.tsx</code>	Bouton manuel "Envoyer aux compagnies" sous un mandat signé
<code>src/components/signatures/ImportDocumentForSignatureDialog.tsx</code>	Dialog broker pour uploader un PDF à signer
<code>src/components/signatures/PdfZonePicker.tsx</code>	Canvas pdfjs-dist multi-page + drag rectangle de zone signature
<code>src/hooks/useMandatDispatch.tsx</code>	Logic dispatch
<code>src/hooks/useClientMandatStatus.tsx</code>	Statut mandat par client

6.8.3 Edge Functions liées

Function	Rôle
<code>send-signature-invite</code>	Envoie email au signataire avec lien <code>https://<tenant>.lyta.ch/signer/:token</code>
<code>proxy-signature-pdf</code>	Proxy le PDF original depuis Storage avec CORS <code>*</code> (contournement Storage CORS pour sous-domaines tenants)
<code>get-signature-pdf-url</code>	Génère un signed URL temporaire pour le PDF
<code>complete-signature</code>	Appelée après signature côté client : merge PDF + signature, store, marque la request <code>signed</code>
<code>dispatch-mandat-to-companies</code>	Envoie le PDF signé à chaque compagnie en email (Resend) + log dans <code>mandat_dispatch_log</code> + <code>tenant_email_log</code>
<code>request-signature-link-renewal</code>	Régénère un token si le lien a expiré (24h)

6.8.4 Tables clés

- `signature_requests` : colonnes critiques
 - `access_token uuid unique` — token public dans l'URL
 - `document_kind` (`mandat_gestion`, `imported`, `autre`)
 - `status` (`draft`, `sent`, `viewed`, `signed`, `cancelled`, `refused`)

- `signature_zone jsonb` — coords normalisées 0-1 (page, x, y, width, height) — ajouté juin 2026
- `preview_file_key` / `signed_file_key` — keys Storage
- `expires_at` — timestamp expiration (par défaut +24h)
- `payload jsonb` — données extras (label, description, originalFileName, etc.)
- `mandat_dispatch_log` : log par compagnie

6.8.5 Business rules clés

- **Token public** : la sécurité du flow repose uniquement sur `access_token` (UUID non-devinable). Le proxy-signature-pdf le vérifie avant de servir le PDF.
- **Expiration** : 24h après création. Renouvelable via `request-signature-link-renewal`.
- **Zone de signature signataire-side** : depuis juin 2026, c'est le **signataire** qui drague la zone via `PdfZonePicker` (canvas pdfjs-dist). Avant, le broker le faisait — inversion documentée et déployée.
- **Document kind = `mandat_gestion`** : rendu HTML (`MandatTemplate`) puis converti en PDF côté client au moment de la signature finale (html2canvas + pdf-lib).
- **Document kind = `imported`** : PDF déjà en Storage, le client le voit via `proxy-signature-pdf` dans `PdfZonePicker`.
- **Dispatch** : manuel par bouton, **pas automatique**. Chaque compagnie de la police du client reçoit un email avec le PDF signé en PJ (sauf si pas d'email mandat configuré). Tracé dans `tenant_email_log` (kind = `mandat_dispatch`).

6.8.6 Gotchas

- **CORS pdfjs** : pdfjs-dist fetch le PDF en cross-origin → Storage refuse les sous-domaines tenants → `proxy-signature-pdf` contourne en CORS `*` (sécurité = token).
- **CSP worker pdfjs** : nécessite `worker-src 'self' blob:` dans le CSP de `index.html`.
- **Worker bundlé local** : `import workerSrc from "pdfjs-dist/build/pdf.worker.min.mjs?url"` — plus de dépendance CDN.
- **Risque juridique mandat avec zone draguable** : Habib a choisi explicitement la zone draguable pour le mandat (juin 2026) malgré l'avertissement sur le risque de contestation (consentement non manifeste si signature pas sous le paragraphe d'engagement). À documenter dans le PV de mission du dev.
- **Type-gen Supabase** : `get_signature_request_by_token` et `mark_signature_request_viewed` ne sont pas dans les types auto-générés → cast `as any` ponctuel dans `Signer.tsx`. **À régénérer.**

6.9 Module 9 — CRM Publicité (Emailing)

6.9.1 Rôle métier

Centre de communication sortante du tenant : - Emails **transactionnels** (invitations signature, confirmations mandat, notifications) - Emails de **campagne** (newsletter, anniversaires) - **Historique unifié** de tous les emails envoyés (vue par filtre kind, status, destinataire) - Templates personnalisables par tenant

Le nom “Publicité” est **trompeur** — c'est en réalité le module **Communications**. Renommage évoqué.

6.9.2 Surface code

Fichier	Rôle
<code>src/pages/crm/CRMPublicite.tsx</code>	Page principale (116 LOC — wrapper avec onglets)
<code>src/components/crm/publicite/EmailDeliveryHistory.tsx</code>	Liste paginée de <code>tenant_email_log</code> avec filtres kind + status
<code>src/components/crm/emails/EmailHistory.tsx</code>	Historique des <code>scheduled_emails</code> (campagnes planifiées) — legacy partiel
<code>src/hooks/useCrmEmails.tsx</code>	Helpers envoi email custom (formulaire “envoyer email à un client”)
<code>src/hooks/useEmailAutomation.tsx</code>	Config automations (anniversaires, échéances polices, etc.)

6.9.3 Edge Functions liées

Function	Rôle
<code>send-crm-email</code>	Envoi email custom depuis CRM (formulaire libre)
<code>send-client-notification-email</code>	Notif client (transactionnel)
<code>send-birthday-emails</code>	Cron quotidien : envoie un email à chaque client dont c’est l’anniversaire
<code>send-renewal-reminders</code>	Cron : alerte client + broker quand une police arrive à échéance
<code>send-follow-up-reminders</code>	Cron : rappels de suivis (tâches non terminées)
<code>send-test-tenant-emails</code>	Outil debug KING : envoie un email test pour valider la config Resend du tenant
<code>process-scheduled-emails</code>	Worker cron : process les <code>scheduled_emails</code> due

6.9.4 Tables clés

- `tenant_email_log` (source de vérité historique) — kinds : `signature_invite`, `mandat_signed`, `mandat_dispatch`, `account_created`, `campaign`, `quick_email`, `crm_email`, `lpp_search`, `transactional`
- `scheduled_emails` (campagnes planifiées — legacy partiel)
- `email_templates` (templates customisables)
- `tenant_email_automation` (config triggers)

6.9.5 Business rules clés

- **Resend** = provider unique (`RESEND_API_KEY`). Identité expéditeur par tenant (`sender_name` + `reply_to` configurés dans `tenant_app_settings`).
- **Tous les emails** loggués dans `tenant_email_log` avec `resend_message_id` pour cross-référence avec les logs Resend.

- **Domaine custom** : si le tenant a configuré un domaine custom (ex: `noreply@advisy.ch`), l'envoi se fait depuis ce domaine via DNS Resend.

6.9.6 Gotchas

- **Deux composants historique** parallèles (`EmailHistory.tsx` lit `scheduled_emails`, `EmailDeliveryHistory.tsx` lit `tenant_email_log`). Le second est la **vraie source de vérité**. Le premier doit être déprécié ou consolidé.
- **Bounces / rejets Resend** : status `bounced` est tracé mais pas de **retry policy automatique**. À implémenter.

6.10 Module 10 — CRM Suivis (Tasks/Reminders)

6.10.1 Rôle métier

Système de **tâches métier** liées à un client : suivi d'une activation de contrat, gestion d'une annulation, traitement d'un sinistre, retour de décompte, résiliation... Plus large qu'un simple TODO : chaque suivi est typé et lié à un workflow.

6.10.2 Surface code

Fichier	Rôle
<code>src/pages/crm/CRMSuivis.tsx</code>	Page principale (306 LOC) — liste, filtres, kanban ?
<code>src/hooks/useSuivis.tsx</code>	Query + CRUD + <code>recordAuditLog</code> sur changements

6.10.3 Types et statuts

```
SuiviType = "activation" | "annulation" | "retour" | "resiliation" | "sinistre" | "autre"
SuiviStatus = "ouvert" | "en_cours" | "ferme"
```

6.10.4 Business rules clés

- **Assigned agent** : chaque suivi est assigné à un collaborateur (`assigned_agent_id`). Scope-aware visibility (agent voit ses propres suivis).
- **Audit logué** : chaque modification déclenche un `recordAuditLog` dans `audit_logs`.

6.10.5 Gotchas

- Module relativement simple, peu de pièges. **Possible amélioration** : kanban drag-drop, notifications push quand un suivi change de status.

6.11 Module 11 — CRM Rapports

6.11.1 Rôle métier

Dashboard analytique du cabinet : KPIs, production, commissions, top clients, top agents, evolution mensuelle, comparaisons N/N-1. Avec génération de rapports custom exportables (PDF/XLSX).

6.11.2 Surface code

Fichier	Rôle	Volume
<code>src/pages/crm/CRMRapports.tsx</code>	Page principale (multi-rapports configurables)	1075 LOC
<code>src/pages/crm/CRMDashboard.tsx</code>	Dashboard d'accueil CRM (KPIs synthétiques)	—

6.11.3 Tech

- **Recharts** pour les graphes (BarChart, LineChart, PieChart)
- **xlsx** pour exports Excel
- **html2canvas + pdf-lib** pour exports PDF (à confirmer)

6.11.4 Business rules clés

- **Scope-aware KPIs** : un agent voit ses propres chiffres, un manager voit ceux de son équipe, un admin voit le cabinet entier.
- **Comparaison N/N-1** : calculs en SQL via window functions (à vérifier).
- **Performance** : sur gros volumes, les queries d'agrégation peuvent être lourdes → cache React Query important.

6.11.5 Gotchas

- **Pas de query cancellation** côté React : si l'utilisateur change de rapport pendant qu'une lourde requête tourne, la première continue à consommer. À surveiller.

6.12 Module 12 — CRM Collaborateurs & Abonnement

6.12.1 Rôle métier

Gestion de l'**équipe du cabinet** + abonnement Stripe : - CRUD collaborateurs (Admin uniquement) - Assignation rôle/scope/branche - Permissions granulaires par collaborateur (override du rôle) - Photo de profil - Adresse pro / horaires - **Seats Stripe** : ajout/retrait de seats facturé automatiquement - Vue abonnement courant (plan, prochaine facture, factures passées)

6.12.2 Surface code

Fichier	Rôle	Volume
<code>src/pages/crm/CRMCollaborateurs.tsx</code>	Page liste + édition	347 LOC
<code>src/pages/crm/CRMAbonnement.tsx</code>	Page abonnement + facturation	467 LOC
<code>src/components/crm/CollaborateurForm.tsx</code>	Form création/édition collab	
<code>src/components/crm/CollaboratorPermissionsDialog.tsx</code>	Dialog permissions granulaires	
<code>src/components/crm/CollaboratorPhotoUpload.tsx</code>	Upload photo	
<code>src/hooks/useCollaborateurs.tsx</code>	Query collab	
<code>src/hooks/useCollaboratorPermissions.tsx</code>	Permissions par collab	
<code>src/hooks/useTenantSeats.tsx</code>	Quota seats	

6.12.3 Edge Functions liées

Function	Rôle
<code>add-user-seat</code>	Stripe : ajoute un seat à l'abonnement courant + crée le collab
<code>create-collaborator</code>	Création collab + invitation email
<code>create-checkout-session</code>	Stripe Checkout pour nouveau plan / upgrade
<code>get-checkout-session-info</code>	Récupère info session Stripe (post-paiement)
<code>list-tenant-invoices</code>	Liste factures Stripe du tenant
<code>cancel-tenant-subscription</code>	Annulation abonnement
<code>sync-tenant-stripe</code>	Resync data Stripe ^[?] Supabase
<code>apply-monthly-overage</code>	Cron mensuel : facture les overages (seats supplémentaires, scans IA sup, etc.)
<code>stripe-webhook</code>	Webhook Stripe (<code>invoice.paid</code> , <code>subscription.updated</code> , etc.)
<code>sync-external-billing</code>	Sync facturation externe (apps tierces connectées)

6.12.4 Business rules clés (critique)

- **Règle facturation fondamentale** : `extra_users` = collaborateurs CRM uniquement. **Les clients du portail espace-client sont illimités et GRATUITS** (cf. `lyta_pricing_users.md` | mémoire projet).

- **Stripe price IDs** : hardcodé en fallback dans `add-user-seat/index.ts` (`FALLBACK_USER_PRICE_ID = "price_1SmZtZF7ZITS358Au3FHsdBA"`). Idéalement à déplacer en `tenant_app_settings` .
- **Proration** : ajout de seat en cours de mois → prorata automatique Stripe.

6.12.5 Gotchas

- **Webhook signature verification** : `stripe-webhook` doit vérifier la signature avec `STRIPE_WEBHOOK_SECRET` — à auditer.
- **Synchronisation Stripe** ^[?] **Supabase** : pas toujours real-time, peut décaler de quelques secondes après un webhook. UI affiche un loader.

6.13 Module 13 — CRM LytaTools

6.13.1 Rôle métier

Module pilote d’apps tierces connectées au tenant. Restreint à **Advisy uniquement** (“Pilot feature restricted to Advisy tenant until validation” — commentaire dans le code).

Permet d’enregistrer des **External Apps** (intégrations vers outils externes type Zapier, calculateur de prime, API tierce...) et de les piloter depuis LYTA.

6.13.2 Surface code

Fichier	Rôle
<code>src/pages/crm/CRMLytaTools.tsx</code> (414 LOC)	Page principale
<code>src/hooks/useLytaTools.tsx</code>	CRUD apps connectées
Table : <code>external_apps</code> , <code>user_app_connections</code>	DB

6.13.3 Business rules clés

- **Gating tenant** : la page redirige si le tenant n’est pas Advisy.
- **Connexion OAuth-like** : flow à confirmer dans le code détaillé.

6.13.4 Gotchas

- Feature **pilote** : pas prête pour la prod publique. **À garder comme telle ou consolider** selon usage Advisy.

6.14 Module 14 — LPP Search

6.14.1 Rôle métier

Recherche LPP (2e pilier suisse) pour un client : envoi de demandes officielles aux deux institutions centralisatrices pour retrouver les avoirs de prévoyance professionnelle d’une personne.

Workflow métier important quand un client change d’employeur ou cherche à consolider ses avoirs (libre passage).

6.14.2 Edge Functions liées

Function	Rôle
<code>send-lpp-search-requests</code>	Envoie automatiquement 2 emails : (1) Centrale du 2e pilier / Sicherheitsfonds BVG, (2) Fondation Institution Supplétive LPP. PJ : pièce d'identité + procuration.

6.14.3 Tables

- `lpp_search_requests` : log des demandes envoyées + statuts retours

6.14.4 Business rules clés

- **2 emails séparés** sont nécessaires (les deux institutions ne se parlent pas).
- **Pièces jointes obligatoires** : pièce d'identité + procuration (mandat de recherche) signée par le client.
- **Identité expéditeur** : email envoyé avec l'identité du tenant (`sender` + `reply_to` du cabinet) — fix juin 2026 (`46e32fe`).

6.14.5 Gotchas

- **Délai de réponse** : les institutions répondent par courrier postal ou email sous 4-8 semaines. Pas de webhook → suivi manuel.

6.15 Module 15 — Comparateur d'assurances

6.15.1 Statut

⚠ **Le comparateur d'assurances n'est PAS dans le repo LYTA.** Il vit dans un projet séparé **Optimis** (cf. mémoire projet `le_comparateur_optimis.md`, `optimis_distribution_canaux.md`).

Relation : Optimis = comparateur Lovable + dashboard de distribution (leads → cabinets distribués). Il **alimente LYTA en leads** via deux canaux : 1. LYTA API (pour les locataires de LYTA) 2. Google Sheets (méthode actuelle pour distribués externes)

Pour le dev : si la mission inclut le comparateur, il faut accéder à un autre repo (`~/Projects/Optimis/`).

6.16 Module 16 — Espace Client (Portal)

6.16.1 Rôle métier

Portail destiné aux **clients finaux du cabinet**. Accès lecture (+ quelques écritures limitées) à leur dossier : - Vue de leurs polices d'assurance - Vue de leurs sinistres - Vue de leurs documents (téléchargement) - Messages avec leur conseiller - Notifications push - Signature de documents (via `/signer/:token`) - Programme de parrainage (`ClientReferrals`)

URL : `<tenant>.lyta.ch/client/*`.

6.16.2 Surface code

Fichier	LOC
src/pages/client/ClientDashboard.tsx	462
src/pages/client/ClientLayout.tsx	482
src/pages/client/ClientDocuments.tsx	538
src/pages/client/ClientContracts.tsx	436
src/pages/client/ClientClaims.tsx	452
src/pages/client/ClientMessages.tsx	440
src/pages/client/ClientReferrals.tsx	352
src/pages/client/ClientProfile.tsx	236
src/pages/client/ClientNotifications.tsx	126
Total	3524

Composant client	Rôle
src/components/client/ClaimForm.tsx	Déclaration sinistre
src/components/client/ ClientNotificationBell.tsx	Bell notifications
src/components/client/MobileBottomNav.tsx	Nav mobile (UX dédiée smartphone)

6.16.3 Authentification client final

Découvert lors de l'audit : - `ClientLayout.tsx:88` contient un commentaire : “*Check if user has a client record (needed for client portal)*” - → Le client utilise le **même** `supabase.auth` que les collaborateurs (table `auth.users`) - → La **distinction** se fait via une **jointure** `auth.users.id` `clients.user_id` lors du chargement du layout - → Si `clients.user_id = current auth.uid()` existe → accès portal. Sinon → redirection.

Conséquence : un même compte Supabase peut être à la fois collaborateur d'un cabinet ET client d'un autre cabinet (cas hypothétique d'un agent qui est aussi client d'un confrère).

6.16.4 Edge Functions liées

Function	Rôle
send-client-message	Envoi message dans le fil de discussion broker [?] client
send-client-notification-email	Notification email au client (nouveau doc disponible, etc.)
submit-referral	Soumission d'un nouveau filleul par le client

6.16.5 Business rules clés

- **Lecture seule** sur les polices, sinistres, commissions. Le client ne peut pas modifier ses contrats.

- **Documents** : peut uploader (pour transmettre au broker) et télécharger (consulter ses pièces).
- **Sinistres** : peut **déclarer** un sinistre via `ClaimForm`.
- **Notifications** : `client_portal_upload` = source des notifs envoyées au broker quand le client upload un doc (source: "client_portal_upload" dans `ClientDocuments.tsx:162`).

6.16.6 Gotchas

- **RLS clients vs auth** : nécessite des politiques spécifiques sur `policies`, `documents`, `claims` qui autorisent `client.user_id = auth.uid()` en plus du chemin collaborateur. À auditer en détail (Doc 2 confidentiel).
- **Pas de MFA SMS** pour les clients finaux à date (à confirmer).

6.17 Module 17 — KING (admin plateforme)

6.17.1 Rôle métier

L'espace administrateur global de la plateforme LYTA, réservé à Habib (Optimislink). C'est le **module le plus volumineux** : 10 171 LOC en pages KING + 16 composants spécifiques.

Fonctions : - **Tenants** : liste, création, activation, suspension, suppression, impersonate - **Onboarding wizard** : flow guidé de création tenant (KingWizard.tsx, 1158 LOC) - **Users** : recherche cross-tenant, gestion comptes - **Plans & quotas** : configuration des plans SaaS, modules par plan, quotas par plan - **Catalogue** : compagnies + produits + branches (catalogue cross-tenant) - **Affiliés** : programme d'affiliation - **Apps externes** : intégrations - **Stripe stats** : MRR, churn, expansion - **Coûts** : monitoring des coûts (Supabase, Resend, OpenAI...) - **Monitoring** : live feed des actions sur la plateforme - **Notifications** : inbox KING (nouvelles inscriptions, alertes, support tickets) - **Sécurité** : config sécurité plateforme (IP allowlist, MFA forcée...) - **Support** : tickets de support des tenants - **Settings** : config globale plateforme - **Compliance report** : rapport de conformité (GDPR, nLPD) - **Tenant import** : import bulk de tenants existants (migration depuis autre CRM)

6.17.2 Surface code (extraits)

Fichier	LOC
<code>src/pages/king/KingWizard.tsx</code>	1158
<code>src/pages/king/KingTenants.tsx</code>	637
<code>src/pages/king/KingAffiliates.tsx</code>	656
<code>src/pages/king/TenantOnboarding.tsx</code>	553
<code>src/pages/king/KingAffiliateDetail.tsx</code>	515
<code>src/pages/king/KingUsers.tsx</code>	366
(autres pages KING)	~6286 cumulés

Composants KING	Rôle
<code>AffiliateInvoiceTab</code>	Factures affiliés
<code>CompanyCatalogManager</code> / <code>ProductCatalogManager</code>	Gestion catalogue cross-tenant
<code>KingLiveFeedCard</code>	Feed temps réel des events plateforme
<code>KingNotificationsInbox</code>	Inbox notifs Habib
<code>KingProductsByBranchCard</code>	Vue produits par branche
<code>OnboardingNotifications</code>	Notifs liées onboarding
<code>PendingProductsManager</code>	Produits détectés par Smartflow en attente de validation
<code>PendingSignupsPanel</code>	Signups Stripe en attente de provisioning
<code>SwissPostalCodesManager</code>	Gestion CP suisses (lookup OpenPLZ)
<code>TenantConsumptionLimits</code> / <code>TenantConsumptionRow</code>	Quotas et conso
<code>TenantDataImport</code> / <code>TenantDocumentImport</code>	Imports massifs
<code>TenantInvoicesPanel</code>	Factures Stripe par tenant
<code>TenantLogoUpload</code>	Upload logo tenant

6.17.3 Edge Functions KING-only

Function	Rôle
<code>activate-tenant</code>	Active un tenant en attente
<code>delete-tenant</code>	Suppression complète (RGPD)
<code>delete-user-account</code>	Suppression user
<code>reset-tenant-data</code>	Reset data tenant (test/migration)
<code>king-impersonate-tenant</code>	Impersonate un tenant (auth temporaire) — à auditer avec attention pour traçabilité
<code>king-stripe-stats</code>	Agrégats Stripe
<code>export-tenant-data</code>	Export RGPD/portabilité données tenant
<code>create-tenant-admin</code>	Création admin tenant (KING)
<code>verify-partner-email</code>	Validation email partenaire
<code>check-slug-availability</code>	Vérif disponibilité slug avant création

6.17.4 Business rules clés

- **Auth KING** : `requireAuth()` puis vérification rôle KING dans `_shared/auth.ts`. Toutes les fonctions KING-only check ça.

- **Audit cross-tenant** : chaque action KING loggée dans `king_audit_logs` (et `king_audit_log` — doublon à consolider).
- **Impersonate** : crée une session temporaire pour Habib dans le contexte d'un tenant. **Critique** : doit être tracé dans audit ET visible côté tenant ("KING a accédé à votre compte le X").

6.17.5 Gotchas

- `king_audit_log` vs `king_audit_logs` : doublon réel — **à consolider en une seule table**. Migration de données nécessaire.
- **KingWizard.tsx 1158 LOC** : monolithique, dur à maintenir. Candidat à découpage en sous-composants.
- **Impersonate sans expiration auto** ? À auditer dans `king-impersonate-tenant`.

6.18 Module 18 — Affiliés

6.18.1 Rôle métier

Programme d'**affiliation LYTA** : apporteurs externes qui amènent des cabinets sur la plateforme et touchent une commission récurrente (% de l'abonnement Stripe sur N mois).

6.18.2 Surface code

Fichier	Rôle	LOC
<code>src/pages/king/KingAffiliates.tsx</code>	Liste affiliés (admin KING)	656
<code>src/pages/king/KingAffiliateDetail.tsx</code>	Détail + factures affilié	515
<code>src/components/king/AffiliateInvoiceTab.tsx</code>	Onglet factures	
<code>src/hooks/useAffiliates.tsx</code>	Query affiliés	

6.18.3 Tables clés

- `affiliates` : profil affilié (nom, prénom, email, IBAN, statut)
- `affiliate_commissions` : commissions calculées sur chaque facture Stripe d'un tenant amené
- `partners` : partenaires commerciaux (peuvent être affiliés ou non)

6.18.4 Edge Functions liées

Function	Rôle
<code>submit-referral</code>	Création d'un filleul (depuis Espace Client)
<code>receive-tenant-request</code>	Réception d'une demande tenant (depuis form public)

6.18.5 Business rules clés

- **Tracking** : un cabinet créé avec `referral_code` est attribué à l'affilié pour les commissions.

- **Commissions affiliés** : déclenchées sur webhooks Stripe `invoice.paid` (à confirmer).
- **Païement affiliés** : via QR-facture suisse générée mensuellement.

6.18.6 Gotchas

- **Pas de portal affilié** dédié à date — les affiliés voient leurs stats via un export ou un email. Peut être amélioré.

✅ **Modules 1-18 documentés.** Tous les modules métier de LYTA ont maintenant une fiche de référence. Les sections 7-13 du document maître (Edge Functions catalogue, Auth détail, Déploiement, Conventions, Dette technique) sont à étoffer en passes suivantes.

7. Edge Functions catalogue

Toutes les 59 edge functions sont listées ci-dessous, groupées par domaine fonctionnel. Pour chacune : rôle, niveau d'auth, et notes critiques.

7.1 Convention `verify_jwt`

Dans `supabase/config.toml`, chaque fonction a un flag `verify_jwt = true|false` : - `true` (défaut) : Supabase rejette l'appel si pas de JWT valide. Le frontend doit fournir le token user via header `Authorization`. - `false` : la fonction reçoit l'appel sans pre-check JWT — c'est à elle de gérer l'auth en interne (via `requireAuth()` du `_shared`, ou par un secret applicatif type Stripe session, ou de manière publique en s'appuyant sur un token UUID).

Toutes les fonctions LYTA sont actuellement en `verify_jwt = false` dans `config.toml` (cf. inspection du fichier). La sécurité est donc déléguée à chaque fonction individuellement via : - `_shared/auth.ts` → `requireAuth()` qui valide le JWT manuellement, et `requireTenantAccess()` qui vérifie l'appartenance au tenant cible. - Tokens publics non-devinables (UUIDs) pour les flows anonymes (signature, deposit-contract). - Secrets de session Stripe pour le self-signup. - Restriction `service_role` only pour les crons.

⚠️ **Implication pour audit** : il faut lire le code de chaque fonction pour savoir si elle valide bien l'auth ou si elle est exploitable. Voir Doc 2 confidentiel.

7.2 Catalogue annoté

Auth & comptes (10 fonctions)

Function	Rôle	Auth interne
<code>create-user-account</code>	Crée un user (utilisé pour invitations collab ou client)	service_role
<code>create-collaborator</code>	Création collab + invitation email	<code>requireAuth</code> admin/manager
<code>create-tenant-admin</code>	Crée admin du tenant + magic link bienvenue Resend	service_role (appelé par provision)
<code>delete-user-account</code>	Suppression user (RGPD)	KING-only
<code>send-password-reset</code>	Reset password via Resend	Public (rate-limit IP)
<code>send-verification-sms</code>	Envoie code SMS Twilio Verify	Public (rate-limit)
<code>verify-sms-code</code>	Valide code SMS et complète l'auth	Public (le code lui-même est le secret)
<code>resend-signup-finalization</code>	KING : relance email "Finalise ton inscription" pour pending_signup orphelin	KING-only
<code>send-sms</code>	Envoi SMS Twilio générique	service_role
<code>verify-partner-email</code>	Validation email partenaire (programme affiliés)	Public via token

Tenants & onboarding (6 fonctions)

Function	Rôle	Auth interne
<code>provision-self-signup-tenant</code>	Orchestrateur self-signup post-Stripe (vérif paiement → crée tenant → admin → DNS → Resend)	Public (secret = Stripe session_id)
<code>check-slug-availability</code>	Vérif live disponibilité slug	Public (rate-limit IP)
<code>get-checkout-session-info</code>	Récupère info session Stripe pour pré-remplir form	Public (secret = session_id)
<code>create-checkout-session</code>	Crée Stripe Checkout (essai 7j)	Public
<code>tenant-onboarding</code>	Setup DNS Cloudflare + Vercel + Resend pour un tenant. Aussi : email DNS records au tenant pour domaine custom	service_role (appelé par provision ou KING)
<code>activate-tenant</code>	Active un tenant en attente	KING-only
<code>delete-tenant</code>	Suppression complète tenant	KING-only
<code>reset-tenant-data</code>	Reset data tenant (test)	KING-only
<code>export-tenant-data</code>	Export RGPD/portabilité	KING-only ou admin tenant

Stripe & facturation (8 fonctions)

Function	Rôle	Auth interne
<code>stripe-webhook</code>	Réception webhooks Stripe (invoice.paid, subscription.*)	Signature Stripe vérifiée
<code>add-user-seat</code>	Ajout seat à l'abonnement Stripe	<code>requireAuth</code> admin tenant
<code>cancel-tenant-subscription</code>	Annulation self-service (cancel_at_period_end)	<code>requireAuth</code> admin tenant
<code>list-tenant-invoices</code>	List factures Stripe d'un tenant	KING-only
<code>sync-tenant-stripe</code>	Sync Stripe customers/subs ? tenants par email	KING-only (ou auto pour 1 tenant)
<code>sync-external-billing</code>	Pull usages réels Resend + Twilio dans <code>platform_usage_logs</code>	KING-only
<code>apply-monthly-overage</code>	Cron : crée invoice items Stripe pour overages	service_role / cron
<code>king-stripe-stats</code>	Agrégats MRR / churn / expansion	KING-only

Signatures (6 fonctions)

Function	Rôle	Auth interne
<code>send-signature-invite</code>	Envoie email avec lien <code>/signer/:token</code>	<code>requireAuth</code> membre tenant
<code>proxy-signature-pdf</code>	Re-stream PDF depuis Storage avec CORS <code>*</code>	Public (auth = access_token query)
<code>get-signature-pdf-url</code>	Signed URL temporaire pour PDF	Public (auth = token)
<code>complete-signature</code>	Merge PDF + signature, store, marque <code>signed</code>	Public (auth = token)
<code>dispatch-mandat-to-companies</code>	Envoi mandat signé aux compagnies par email + log	<code>requireAuth</code> membre tenant
<code>request-signature-link-renewal</code>	Notifie le broker quand le client demande un nouveau lien (token expiré)	Public (auth = token) — n'auto-renouvelle PAS (sinon faille)

Documents & Smartflow (3 fonctions)

Function	Rôle	Auth interne
<code>scan-document</code>	OCR + classification IA d'un doc unitaire	<code>requireAuth</code> + quota
<code>classify-batch-documents</code>	Traitement lot avec quotas	<code>requireAuth</code> + <code>reserveTenantQuota</code>
<code>scan-commission-statement</code>	OCR décompte gpt-5 + parsing lignes + match auto	<code>requireAuth</code> + quota

Polices & contrats (2 fonctions)

Function	Rôle	Auth interne
<code>save-policy</code>	INSERT/UPDATE police avec UUID forcé backend	<code>requireAuth</code> membre tenant
<code>deposit-contract</code>	Dépôt public d'un contrat par un client	Public (form public)
<code>send-contract-deposit-email</code>	Notif au broker après dépôt	service_role

Clients & fallbacks RLS (2 fonctions)

Function	Rôle	Auth interne
<code>create-client</code>	Fallback INSERT client (bypass RLS 42501)	<code>requireAuth</code> + check membre tenant + perm <code>clients.create</code>
<code>bypass-insert</code>	Fallback INSERT générique (family_members + documents)	<code>requireAuth</code> + check tenant + whitelist tables

Emails transactionnels & cron (9 fonctions)

Function	Rôle	Auth interne
<code>send-crm-email</code>	Email custom CRM	<code>requireAuth</code> membre tenant
<code>send-client-message</code>	Message broker [?] client	<code>requireAuth</code>
<code>send-client-notification-email</code>	Notif email client	service_role
<code>send-claim-notification</code>	Notif sinistre	service_role
<code>send-birthday-emails</code>	Cron quotidien : anniversaires clients	service_role / cron
<code>send-renewal-reminders</code>	Cron quotidien : échéances polices	service_role / cron
<code>send-follow-up-reminders</code>	Cron quotidien : relance prospects dormants	service_role / cron
<code>send-test-tenant-emails</code>	KING : envoie email test de chaque template	KING-only
<code>process-scheduled-emails</code>	Worker cron : <code>scheduled_emails</code> due	service_role

LPP & assurance suisse (4 fonctions)

Function	Rôle	Auth interne
<code>send-lpp-search-requests</code>	Envoi 2 emails aux institutions LPP (Sicherheitsfonds + Auffangeinrichtung)	<code>requireAuth</code> membre tenant
<code>swiss-postal-code-lookup</code>	Proxy OpenPLZ Suisse	Public (rate-limit)
<code>swiss-address-lookup</code>	Proxy swisstopo SearchServer (autocomplete adresses)	Public (rate-limit)
<code>health-check</code>	Healthcheck plateforme	Public

IA & assistant (1 fonction)

Function	Rôle	Auth interne
<code>ai-chat</code>	Assistant IA conversationnel (chat)	<code>requireAuth</code> + quota

KING admin (1 fonction sensible)

Function	Rôle	Auth interne
<code>king-impersonate-tenant</code>	Impersonate : magic link de connexion comme admin d'un tenant. Loggé <code>king_audit_log</code> <code>action_type='tenant.impersonate'</code> .	KING-only

Affiliés & onboarding partenaires (2 fonctions)

Function	Rôle	Auth interne
<code>submit-referral</code>	Soumission filleul (depuis Espace Client)	Public via auth client
<code>receive-tenant-request</code>	Réception demande tenant (form public)	Public

8. Auth & Permissions (détail technique)

8. Auth & Permissions (détail technique)

8.1 Supabase Auth (GoTrue)

- **Backend** : géré par Supabase (GoTrue). Pas de code custom côté Auth strictement parlant.
- **Provider** : email + password uniquement (pas de OAuth Google/Microsoft à date).
- **JWT** :
 - Algorithme HS256
 - Secret : `SUPABASE_JWT_SECRET` (côté Supabase)
 - Expiration : **3600s (1h)** (`jwt_expiry = 3600` dans `config.toml`)
 - Refresh : automatique côté client supabase-js via `refresh_token`
- **Storage des sessions côté client** :
 - localStorage par défaut (`supabase-js`)
 - Un second client (`smsClient`) est instancié pour la phase MFA SMS avec sa propre storage isolée (sessionStorage) — voir `useAuth.tsx`
- **Magic links** : `otp_expiry = 86400` (24h, augmenté depuis 1h), `otp_length = 8`.

8.2 MFA

- **TOTP** : `enroll_enabled = true`, `verify_enabled = true` dans `config.toml`. Max 10 facteurs.

- **SMS** : `enroll_enabled = false`, `verify_enabled = false` côté Supabase Auth standard. **LYTA implémente une MFA SMS custom** via Twilio Verify (`send-verification-sms` + `verify-sms-code`), pas via le mécanisme natif Supabase.

8.3 RLS (Row Level Security)

Toutes les tables `public.*` ont RLS activé. Les politiques utilisent des **fonctions helper SECURITY DEFINER** pour éviter la récursion et hoister les calculs :

Fonction	Rôle
<code>get_user_tenant_id()</code>	Retourne le <code>tenant_id</code> actif depuis le JWT ou <code>user_tenant_roles</code>
<code>my_collab_id_for_active_tenant()</code>	Retourne le collaborator id de l'user courant
<code>has_role(role_name)</code>	Check rôle global
<code>is_crm_member_of_tenant(tenant_id)</code>	Check appartenance tenant
<code>can_access_client(client_uuid)</code>	Compose les checks pour autoriser l'accès à un client
<code>_client_is_in_team(client_uuid, manager_uuid)</code>	Pour scope team (sans récursion)

Détail des politiques par table critique → **Doc 2 confidentiel**.

8.4 Sessions

- **Timeout d'inactivité** : configurable par tenant via `tenant_security_settings.session_timeout_minutes`. Géré côté client par `useSessionTimeout.ts` qui écoute les events souris/clavier.
- **Logout forcé** : déclenché par `useForcedLogoutAfter` après des actions critiques (changement de rôle, MFA enrôlée, mot de passe changé).
- **statement_timeout** Postgres : 30s pour le rôle `authenticated` (`ALTER ROLE authenticated SET statement_timeout = '30s'`). Coupe les queries qui partent en runaway.

8.5 Permissions granulaires

Au-delà du rôle (`admin` / `manager` / `agent`), `collaborator_permissions` permet d'**override** finement les droits d'un collab : - `clients.create`, `clients.delete` - `policies.write`, `policies.export` - `commissions.see_amounts`, `commissions.validate` - etc.

Le hook `usePermissions()` agrège rôle + permissions + flags du plan pour exposer `can(action)`.

9. Déploiement

9. Déploiement

9.1 Frontend (Vercel)

- **Auto-deploy** sur push GitHub branche `main` (LYTA-git privé).
- **Preview deploys** sur PR.
- **Domaines** :
 - `app.lyta.ch` (principal)
 - `*.lyta.ch` (sous-domaines tenants ajoutés dynamiquement via `tenant-onboarding`)
 - `lyta-xi.vercel.app` (URL Vercel par défaut, backup)
- **Build** : `npm run build` → `dist/` → CDN Vercel.
- **Env vars** côté Vercel (à confirmer dans le dashboard) :
 - `VITE_SUPABASE_URL`
 - `VITE_SUPABASE_ANON_KEY`

9.2 Backend (Supabase CLI manuel)

⚠ **Pas d'auto-deploy backend.** Toutes les opérations backend sont **manuelles**, lancées depuis le poste de Habib avec `supabase` CLI.

Migrations DB

```
# Lister les migrations à appliquer
supabase db diff --schema public

# Appliquer en remote
supabase db push --linked

# Reset local (sandbox de dev)
supabase db reset
```

Les migrations vivent dans `supabase/migrations/*.sql` (256 à date), nommées avec timestamp croissant `YYYYMMDDHHMMSS_<description>.sql`.

Edge Functions

```
# Déployer une fonction
supabase functions deploy <name>

# Toutes les fonctions
supabase functions deploy

# Set un secret
supabase secrets set RESEND_API_KEY=re_XXX
```

Storage policies

Configurées via migrations SQL (`storage.objects` policies).

9.3 Workflow de déploiement (cf. mémoire Habib)

- **Frontend** : push GitHub → Vercel build auto (~1-2 min) → live.
- **Backend** : commit local → `supabase db push` ou `supabase functions deploy <name>` manuel.
- **Règle Habib** : “Never push to GitHub without explicit approval” — toujours commit local d’abord, puis push uniquement après validation.

9.4 Rollback

Type	Procédure
Frontend	Vercel : “Redeploy” sur un commit précédent depuis le dashboard
Edge function	<code>supabase functions deploy <name></code> en repointant sur le précédent code (Git)
Migration DB	Pas de rollback automatique — il faut écrire une nouvelle migration de réversion. Toutes les migrations sont conçues pour être idempotentes / additives.

9.5 Monitoring & logs

- **Vercel** : logs frontend dans le dashboard Vercel.
- **Supabase** : logs edge functions accessibles via dashboard (panneau “Logs”). Les `createLogger("name")` du `_shared/logger.ts` standardisent les logs JSON structurés.
- **Healthcheck** : `health-check` edge function.
- **Errors front** : pas de Sentry à date — uniquement `console.error` (à améliorer).

10. Conventions code

10. Conventions code

10.1 Structure & naming

- **Pages** : `PascalCase.tsx` (`CRMClients.tsx` , `Signer.tsx`)
- **Composants** : `PascalCase.tsx`
- **Hooks** : `useXxx.tsx` (`useClients` , `useAuth`)
- **Lib utilitaires** : `camelCase.ts` (`edgeFunctions.ts` , `tenantUrls.ts`)

- **Types** : declared inline ou dans `integrations/supabase/types.ts` (auto-généré)

10.2 Patterns React

- **Functional components only** (pas de class components).
- **Hooks** : un hook custom par préoccupation métier. Convention : retourner `{ data, loading, error, ...mutations }`.
- **State serveur** : **TanStack Query** systématique. Pas de `useEffect + fetch` direct.
- **Forms** : `react-hook-form` + `zod` schema.
- **Errors** : `translateError(err)` (`src/lib/errorTranslations.ts`) pour convertir les erreurs Supabase en messages FR.
- **Toasts** : `useToast()` (Radix) ou `sonner` selon le composant.

10.3 Conventions Supabase

- **Query** : `supabase.from('table').select(...).eq(...).maybeSingle()` (préfère `maybeSingle` à `single` pour ne pas planter si 0 row).
- **Edge functions** : invoquées via `invokeSupabaseFunction(name, { body })` (wrapper `src/lib/edgeFunctions.ts`) qui gère uniformément les erreurs et les CORS.
- **Pagination** : `usePaginatedQuery()` hook custom standardisé pour toutes les listes.
- **RLS** : ne jamais désactiver, ne jamais bypass front. Si besoin de bypass → passer par edge function `service_role` (cf. `create-client`).

10.4 i18n

- **Lib** : `react-i18next` (vu dans `ClientDashboard.tsx`).
- **Langues** : FR principalement. EN partiel (à vérifier).

10.5 Tests

- **Statut** : pas de framework de tests automatisés à date.
- **Tests manuels** : documentés dans `Documentation/TESTS_A_REALISER.md`.
- **Recommandation pour le dev** : introduire **Vitest** pour les hooks + **Playwright** pour les flows critiques (signup, signature, dispatch mandat).

10.6 Linting

- **ESLint 9 + typescript-eslint 8**.
- **Plugins** : `react-hooks`, `react-refresh`.
- **Pre-commit** : pas de Husky à date — recommandé d'en ajouter pour empêcher les pushes avec erreurs ESLint/tsc.

11. Dette technique & bugs UX connus

11. Dette technique & bugs UX connus

11.1 Bugs récemment corrigés (juin 2026)

Bug	Fix	Commit
RLS clients timeout 80s (Stéphane JCG)	RLS simple + filtre scope-aware côté frontend dans <code>useClients.tsx</code>	revert + <code>46ms</code>
PDF signature CORS sur sous-domaines tenants	Edge function <code>proxy-signature-pdf</code> en CORS <code>*</code>	<code>e01417e</code>
PDF worker pdfjs bloqué par CSP <code>script-src</code>	Worker bundlé local via Vite <code>?url</code>	<code>93ceef8</code>
Worker blob: bloqué par CSP <code>worker-src</code>	Ajout <code>worker-src 'self' blob:</code> dans CSP	<code>93ceef8</code>
Inversion workflow signature mandat (broker dessinait zone)	Corrigé : zone dessinée par signataire	<code>d7f57ac</code>
Affichage double PDF dans Signer	Un seul PdfZonePicker, suppression <code><object></code>	<code>a7bc6ff</code>
Storage bucket 10MB → 25MB + HEIC/HEIF	Migration <code>20260603210000</code>	—
Family member 403 PostgREST	Edge function <code>bypass-insert</code>	<code>61eeb29</code>
Scan validation client INSERT RLS	Routing via <code>create-client</code>	<code>f25593f</code> , <code>79e2b75</code>
Smartflow Lemanica VIE vs LPP confusion	Distinguer selon contenu réel doc	<code>8853c94</code>
LPP search emails identité tenant	sender + reply_to du tenant	<code>46e32fe</code>
TTL liens invitation/reset 1h → 24h	<code>otp_expiry = 86400</code>	<code>51695a9</code>
<code>save-policy</code> UUID backend + suppr <code>.select()</code> post-INSERT	Évite RLS SELECT immédiat	<code>d76c0a8</code>
Scan emails enrichissement client existant	Match VIE/LPP amélioré	<code>ff9425b</code>

11.2 Dette technique structurelle identifiée

● Critique

- **Bug RLS 42501 récurrent sur INSERT** : touche `clients`, `polices`, `family_members`, `documents`. Workaround via 3 edge functions (`create-client`, `save-policy`, `bypass-insert`). **Cause racine non identifiée** : “mismatch SQL CLI vs PostgREST runtime” selon le commentaire de `create-client/index.ts`. **À investiguer en priorité par le dev** — c’est une dette qui empêche l’optimisation des flows et ralentit l’UX.
- **Type-gen Supabase out-of-sync** : `commission_statement_lines`, `get_signature_request_by_token`, `mark_signature_request_viewed` ne sont pas dans les types auto-générés → casts `as any` éparpillés. Régénérer via `supabase gen types typescript --linked > src/integrations/supabase/types.ts`.

● Modéré

- **Tables doublonnes** :
 - `audit_log` vs `audit_logs` (à vérifier)
 - `king_audit_log` vs `king_audit_logs` (doublon **confirmé** — voir Module 17)
 - **Consolider en migration** avec préservation des données existantes.
- **contracts table legacy** : tout le code utilise `polices`, mais la table `contracts` existe. `save-policy` parle de `module = "contracts"` qui est l’identifiant du module dans `platform_modules`. **À auditer + nettoyer**.
- **clients_safe vue** : présente dans le schéma, usage actuel inconnu. Si plus utilisée → drop.
- **EmailHistory legacy** : `EmailHistory.tsx` lit `scheduled_emails`, `EmailDeliveryHistory.tsx` lit `tenant_email_log`. **Consolider** en un seul composant lisant la table unifiée.
- **frame-ancestors CSP** : délivré via `<meta>` donc ignoré par les browsers. À déplacer en header HTTP Vercel (`vercel.json` headers).
- **KingWizard.tsx 1158 LOC monolithique** : candidate à découpage en sous-composants.

● Mineur

- **Pas de Sentry** : pas de tracking erreurs frontend en prod. Recommandé.
- **Pas de tests automatisés** : Vitest + Playwright recommandés (cf. §10.5).
- `statement_timeout = 30s` : peut couper des exports légitimes (gros tenants). À monitorer.
- **Pas de retry policy** sur les bounces Resend.
- **Pas de query cancellation** dans `CRMRapports.tsx` quand l’user change de rapport.
- **Pas de portal affilié dédié** — affiliés voient stats via export/email.
- **Hardcoded Stripe price ID** dans `add-user-seat` (fallback). À déplacer en `tenant_app_settings`.
- **king-impersonate-tenant** : à auditer pour expiration auto + visibilité côté tenant (“KING a accédé à votre compte”).

11.3 Quick wins suggérés au dev

1. **Régénérer les types Supabase** (~5 min) → supprime les `as any`.

2. **Investiguer le bug RLS 42501** (~1-2 jours) → permettrait de supprimer 3 edge functions de bypass et de revenir à des INSERTs front directs.
3. **Consolider les tables audit doublonnes** (~2-3h avec migration de données).
4. **Déplacer `frame-ancestors` en header Vercel** (~15 min).
5. **Ajouter Sentry** (~1h setup + config).
6. **Mettre en place Husky + ESLint pre-commit** (~30 min).
7. **Cleanup `contracts` legacy table** (audit + drop si pas utilisée, ~1h).

12. Roadmap

Voir le document vivant [Documentation/Roadmap_LYTA_Officielle.md](#) (au format PDF aussi).

Roadmap LYTA officielle = 6 priorités (statut     mis à jour à chaque session).

13. Annexes

13.1 Liste complète des hooks personnalisés (50)

```
useAffiliates, useAgents, useAuth, useCelebration, useClientMandatStatus,
useClientNotifications, useClients, useCollaborateurs, useCollaborateursCommission,
useCollaboratorPermissions, useCommissionParts, useCommissions, useCompanyContacts,
useCrmEmails, useDocuments, useEmailAutomation, useFamilyMembers, useForcedLogoutAfter,
useInsuranceCompanies, useInsuranceProducts, useKingNotifications, useLanguage,
useLytaTools, useMandatDispatch, useNotifications, usePaginatedQuery,
usePasswordCheck, usePendingProducts, usePendingScans, usePerformance, usePermissions,
usePlanFeatures, usePlatformSettings, usePolicies, useProductCatalog, useQRInvoices,
useScanBatches, useSessionTimeout, useStripeStats, useSuivis, useTenantBranches,
useTenantConsumption, useTenantLookups, useTenantRoles, useTenantSeats, useTheme,
useUserRole, useUserTenant, use-mobile, use-toast
```

13.2 Liste complète des edge functions (59)

activate-tenant, add-user-seat, ai-chat, apply-monthly-overage, bypass-insert, cancel-tenant-subscription, check-slug-availability, classify-batch-documents, complete-signature, create-checkout-session, create-client, create-collaborator, create-tenant-admin, create-user-account, delete-tenant, delete-user-account, deposit-contract, dispatch-mandat-to-companies, export-tenant-data, get-checkout-session-info, get-signature-pdf-url, health-check, king-impersonate-tenant, king-stripe-stats, list-tenant-invoices, process-scheduled-emails, provision-self-signup-tenant, proxy-signature-pdf, receive-tenant-request, request-signature-link-renewal, resend-signup-finalization, reset-tenant-data, save-policy, scan-commission-statement, scan-document, send-birthday-emails, send-claim-notification, send-client-message, send-client-notification-email, send-contract-deposit-email, send-crm-email, send-follow-up-reminders, send-lpp-search-requests, send-password-reset, send-renewal-reminders, send-signature-invite, send-sms, send-test-tenant-emails, send-verification-sms, stripe-webhook, submit-referral, swiss-address-lookup, swiss-postal-code-lookup, sync-external-billing, sync-tenant-stripe, tenant-onboarding, verify-partner-email, verify-sms-code

13.3 Glossaire

Terme	Définition
KING	Espace d'administration de la plateforme LYTA (réservé à Optimislink / Habib)
Tenant	Cabinet utilisateur de LYTA (= 1 client SaaS)
Collaborateur	Membre d'un cabinet (admin / manager / agent) — facturé en seat
Client	Personne physique conseillée par un cabinet — non facturée
Mandat de gestion	Document juridique signé par un client autorisant le cabinet à agir en son nom auprès des compagnies
Dispatch	Envoi automatisé d'un mandat signé à toutes les compagnies en PJ
Smartflow	Module de scan + classification IA de documents
LPP	2e pilier suisse (prévoyance professionnelle)
LAMal / LCA	Assurance maladie de base / complémentaire (Suisse)
3a / 3b	3e pilier suisse (prévoyance individuelle liée / libre)
RC	Responsabilité civile
Décompte	Document mensuel/trimestriel des compagnies listant les commissions dues
Rétrocession	Reversement d'une partie de commission à un agent ou apporteur
Seat	Place de collaborateur (unité de facturation)
Slug	Identifiant URL d'un tenant (ex: <code>advisy</code> , <code>jcgconsulting</code>)

13.4 Documents existants dans le repo (à lire en complément)

Le repo contient déjà plusieurs documents qu'il faut consulter en parallèle de celui-ci :

Fichier	Rôle	Statut
<code>README.md</code> (racine)	Démarrage local, env vars Vite, déploiement Vercel	✅ Court mais à jour
<code>Documentation/RECAP_5-18_MAI_2026.md</code>	Récap exhaustif de 14 jours d'activité (154 commits, ~30 features)	✅ Source primaire pour comprendre l'historique récent
<code>Documentation/TESTS_A_REALISER.md</code>	Checklist de tests manuels validés/non-validés	✅ Utilisable comme base d'une suite automatisée
<code>Documentation/legal/CGU.md</code>	Conditions Générales d'Utilisation	⚠️ Contient des [À COMPLÉTER] (raison sociale, IDE, adresse)
<code>Documentation/legal/POLITIQUE_CONFIDENTIALITE.md</code>	Politique RGPD/nLPD	⚠️ Idem [À COMPLÉTER] + références art. 5 nLPD et UE 2016/679
<code>Documentation/legal/MENTIONS_LEGALES.md</code>	Mentions légales	⚠️ Idem [À COMPLÉTER]

🔴 Alerte légale (à corriger AVANT mise en ligne publique)

Les **3 documents légaux** contiennent des placeholders `[À COMPLÉTER]` pour : raison sociale exacte, adresse postale, numéro IDE CHE-, registre du commerce du canton, numéro TVA, capital social, téléphone. **Tant que ces docs ne sont pas finalisés, ils ne peuvent pas être affichés tels quels sur la plateforme** — la publication d'un service en ligne en Suisse impose des mentions légales conformes (art. 322 CO + OF-COM). Habib doit les remplir avec les coordonnées exactes d'Optimislink Sàrl avant la prochaine vague d'inscription externe.

13.5 Détails opérationnels concrets (extraits du code + RECAP)

Cette section consolide les éléments **opérationnels précis** qui vivent dans le code et qui peuvent manquer au dev débutant son audit. Tout est sourcé du RECAP du 5-18 mai 2026 ou de lecture directe du code.

Quotas par plan (Smartflow / SMS / Email auto par mois)

Plan	Smartflow scans	SMS	Email
Start	0	0	0
Pro	0	200	2000
Prime	400	400	10000
Founder	400	400	10000

- Override possible par tenant : ex. **Advisy a son quota Smartflow bumpé à 500/mois** (manuellement)
- `auto_overage_enabled` (boolean sur `tenants`) : si `true`, les dépassements sont auto-facturés en invoice items Stripe via le cron `apply-monthly-overage`
- Widget tenant `TenantQuotaWidget` : 3 jauges Smartflow / SMS / Email
- Alertes 80% / 100% / 150% : email tenant + notif KING

Crons (6 jobs automatisés)

Cron	Fréquence	RPC test immédiat	Filtre
 Anniversaires	quotidien 07:00 Europe/ Zurich	<code>SELECT public.trigger_birthday_emails()</code>	clients status='actif'
 Renouvellements échéance polices	quotidien 07:30	<code>SELECT public.trigger_renewal_reminders()</code>	client actif + police active
 Follow-up prospects dormants	quotidien 08:00	<code>SELECT public.trigger_follow_up_reminders()</code>	status='prospect' seulement
 Retry onboarding	horaire	—	tenants en <code>pending_setup</code>
 Auto-activate trial expiré	horaire	—	tenants en <code>trialing</code> arrivés à échéance
 Apply monthly overage	mensuel	—	tenants avec <code>auto_overage_enabled=true</code>

Setup Vault Supabase requis pour les crons (sinon ils tournent silencieusement sans rien faire) :

```
SELECT vault.create_secret('https://shxbcszukoegvvejcpns.supabase.co', 'PROJECT_URL');
SELECT vault.create_secret('<service_role_key>', 'SERVICE_ROLE_KEY');
```

Toggles d'emails auto (5 toggles dans `tenant_email_automation`)

Configurables par tenant dans CRM → Paramètres → Emails :

1. `enable_welcome_email` — Bienvenue espace client
2. `enable_contract_deposit_email` — Confirmation après dépôt contrat
3. `enable_contract_signed_email` — Contrat signé
4. `enable_mandat_signed_email` — Mandat de gestion signé
5. `enable_account_created_email` — Compte client créé

Plus 3 toggles spécifiques crons : - `enable_birthday_email` - `enable_renewal_reminder` (+ `renewal_reminder_days_before`) - `enable_follow_up_reminder` (+ `follow_up_reminder_days`)

Templates emails testables (8 templates branded)

KING-only : bouton “Tester emails” dans `KingTenantDetail` → envoi d’un seul clic les 8 templates branded à une boîte cible :

1. `welcome` — bienvenue espace client
2. `account_created` — compte client créé

3. `contract_signed` — contrat signé
4. `mandat_signed` — mandat signé
5. `relation_client` — relation client transmise
6. `offre_speciale` — offre commerciale
7. `password_reset` — lien réinitialisation
8. `finalize_signup` — filet post-paiement Stripe

Page `/deposer-contrat` — 5 tabs

Page **publique** (sans auth) avec radio-tabs :

Tab	Branche	Détail
SANA	Maladie de base + complémentaire	Form structuré
VITA	Vie / 3e pilier	Form structuré
MEDIO	Indemnités journalières	Form structuré
BUSINESS	Risques pro	Form structuré
 LPP (ambre)	2e pilier — recherche / libre passage / rapatriement	Smartflow scan en haut + 7 sections dynamiques selon checkbox

LPP particularités : - 3 checkboxes (au moins 1 requise) : recherche d'avois / création libre passage / rapatriement LPP - N° AVS validé format `756.XXXX.XXXX.XX` - Sections "Anciens employeurs" / "Libre passage" / "Caisse pension actuelle" apparaissent dynamiquement - Upload obligatoire : pièce d'identité + facture QR + contrat + procuration - Submit → `policies` créé avec `product_type='lpp'` + email cabinet

Signature anchor (zone fixe historique)

Avant juin 2026 : la zone de signature pour les docs imported avait une grille **3×3** par page (haut-gauche, haut-centre, ..., bas-droite). Le broker choisissait dans le formulaire d'envoi quelle cellule + quelle page.

Depuis juin 2026 : remplacé par `PdfZonePicker` (drag rectangle libre par le signataire). Le code 3×3 reste comme **fallback** dans `Signer.tsx` → `generateSignedPdfBase64` : si `signerZone` est null ET `requestZone` est null, on retombe sur l'anchor 3×3.

Ordre de priorité : `signerZone > requestZone > anchor 3×3`.

Swiss postal autocomplete — 3 fallbacks

1. **OpenPLZ** (`openplz.org`) — primary
2. **Zippopotam** (`api.zippopotam.us`) — fallback 1
3. **swisstopo SearchServer** (`api3.geo.admin.ch`) — fallback 2 (adresses de rue, source officielle swisstopo)

Tous proxifiés via edge functions (`swiss-postal-code-lookup`, `swiss-address-lookup`) pour éviter les soucis CSP/CORS côté client.

Smartflow — 14 types de documents reconnus

Liste dans `classify-batch-documents/DOC_CLASSIFICATIONS` :

- `identity_doc` — pièce d'identité (passeport, carte ID, permis)
- `old_policy` — ancienne police (à résilier/remplacer)
- `policy` — police active
- `mandat` — mandat de gestion
- `fiche_salaire` — fiche de paie
- `attestation_avoirs` — attestation d'avoirs
- `decompte_commission` — décompte de commission
- `facture_qr` — facture QR suisse
- `procuration` — procuration
- (et 5 autres — à confirmer en lisant le code)

Particularités : - **Lemania** : couvre VIE (3e pilier) ET LPP (2e pilier) — distinguer selon le contenu réel du doc (fix `8853c94`) - **Anti-doublons** : par `(company, category)` SAUF 3e pilier (split autorisé entre plusieurs polices) - **Branches taxonomy** : LAMal + LCA = **branche combinée**, legacy 'multirisque' dropé - **Catalog-aware prompt** : matching produits via **fuzzy NFD + trigram** (similarité texte) - **Lazy creation** : rien n'est écrit en DB tant que le broker n'a pas validé dans `ScanValidationDialog` - **Family flow** : N clients + N contrats routés par personne assurée (LAMal + LCA même personne = 1 contrat canonical)

Self-signup — flow détaillé (issu du RECAP)

1. Marketing site lyta.ch (Lovable) : visiteur clique "Commencer mon essai 7 jours"
2. Stripe Payment Link → paiement
3. Webhook Stripe → asynchrone (constructEventAsync pour Deno crypto compat)
4. Email 1 "Active ton cabinet LYTA" (filet sécu)
5. Redirect /finalize?session_id=cs_xxx
6. Form /finalize 4 sections :
 - Entreprise : nom, slug (vérif live via check-slug-availability), logo (PNG/JPEG/WebP/SVG max 2MB)
 - Couleurs : primary + secondary (color picker hex)
 - Contact : admin (prénom/nom/email), email back-office (pour reply-to)
 - Options : collaborateurs CRM supplémentaires (label précise "clients finaux illimités gratuits")
7. Submit → provision-self-signup-tenant
8. Provision : tenant (pending_setup) → admin user (auth.users) → tenant-onboarding step="full"
 - Cloudflare DNS CNAME `<slug>.lyta.ch` → cname.vercel-dns.com`
 - Vercel domain ajouté au projet (API token)
 - Resend audience créée
9. Notifs King en cascade :
 - "🆕 Nouveau paiement self-signup"
 - "🏠 Tenant créé"
 - "✅ DNS configuré"
10. Email 2 welcome + magic link → user clique → /reset-password → /connexion
11. SMS 2FA OTP envoyé via Twilio (enable_2fa_login=true PAR DÉFAUT au self-signup)
12. Tenant actif en <5 min en cas idéal, sinon Habib reçoit alertes et peut "Re-run onboarding"

Bug critique historique évité (à ne pas refaire) : *"mandat-signed wipe admin password"* — un trigger DB sur signature mandat a reset le password admin. Fix appliqué mai 2026. Audit recommandé sur tous les triggers DB qui touchent `auth.users`.

Détails techniques sensibles

- **Phone column** : stocké en **top-level** sur `auth.users.phone` (pas dans `user_metadata`) pour que Twilio OTP fonctionne via Supabase Auth natif.
- **statement_timeout** : 8s historiquement, monté à 30s pour le rôle `authenticated` (`ALTER ROLE authenticated SET statement_timeout = '30s'`).
- **Tests sandbox** : Stripe en `test mode` (clé `sk_test_...`), Twilio en test mode, Resend en test mode pour ne pas spammer.
- **LPP institutions test** : env vars `LPP_CENTRALE_EMAIL` et `LPP_SUPPLETIVE_EMAIL` overridables côté Supabase Dashboard pour pointer vers une boîte test au lieu des vraies institutions (`info@sfbvg.ch` + `kontakt@chaeis.net`).
- **send-sms isolé du auth wrapper** : la fonction `send-sms` est isolée du middleware d'auth principal pour éviter la perte de session pendant l'envoi.
- **constructEventAsync** : utilisé dans `stripe-webhook` au lieu de `constructEvent` synchrone — compat Deno crypto.
- **Sidebar restructure** : 14 → 8 entrées top-level depuis mai 2026 (Onboarding et Paramètres regroupés en sous-menus).

Métriques d'activité (sur la fenêtre 5-18 mai 2026)

- 154 commits sur 14 jours = ~11 commits/jour
- +25 000 lignes ajoutées / ~3 000 supprimées
- 30+ edge functions créées ou modifiées
- 20+ migrations DB
- 6 phases KING Platform livrées
- 0 régression non-fixée à la sortie

13.6 Variables d'environnement requises

Voir le **Doc 2 confidentiel** pour les **valeurs**. Variables attendues :

Frontend (`.env.local` Vite)

- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY`

Edge Functions (Supabase secrets)

- `SUPABASE_URL`
- `SUPABASE_SERVICE_ROLE_KEY`
- `SUPABASE_ANON_KEY`
- `RESEND_API_KEY`
- `RESEND_FROM_EMAIL` (par tenant ou global)
- `TWILIO_ACCOUNT_SID` / `TWILIO_AUTH_TOKEN` / `TWILIO_VERIFY_SID`
- `STRIPE_SECRET_KEY` / `STRIPE_WEBHOOK_SECRET`
- `OPENAI_API_KEY` ou `ANTHROPIC_API_KEY`
- `ALLOWED_ORIGINS` (CSV des origines autorisées pour les fonctions ne forçant pas CORS `*`)

Prochains points d'étape de ce document

Ce document est volontairement livré **en plusieurs passes** :

- **Passe 1 (livrée)** : Sections 1-5 (vue d'ensemble → multi-tenancy) + squelette du reste
- **Passe 2** : Modules métier 1-6 (Auth, Clients, Polices, Commissions, Compagnies, Compta)
- **Passe 3** : Modules métier 7-12 (Smartflow, Signatures, Publicité, Suivis, Rapports, Collaborateurs)
- **Passe 4** : Modules métier 13-18 (LytaTools, LPP, Comparateur, Client portal, KING, Affiliés)
- **Passe 5** : Edge Functions catalogue + Auth détail + Déploiement
- **Passe 6** : Conventions code + Dette technique complète + finalisation

Le Doc 2 confidentiel (`LYTA_Security_Audit_CONFIDENTIAL.md`) sera produit séparément après la Passe 6.

Document généré le 8 juin 2026 par Habib Agharbi (Optimislink Sàrl). Tous droits réservés.